



1-Month Software Development Bootcamp: Python & Flask Coursebook

Introduction

Welcome to the **Beginner Software Development Bootcamp**! This 4-week intensive course will take you from zero coding experience to deploying a full-stack web application using Python and the Flask framework. The course assumes **no prior programming knowledge** – just enthusiasm and a willingness to learn. We will start with the fundamentals of Python programming and gradually introduce web development concepts, leading up to building and deploying your own Flask web app by the end of the bootcamp. Each week is structured with daily objectives, theoretical lessons, hands-on labs, and projects to reinforce your learning:

- **Week 1 – Python Fundamentals:** Learn the basics of programming in Python, including syntax, data types, control flow, and functions. By the end of Week 1, you will write simple programs and a mini-project, and gain familiarity with using the command line (CLI) and Python tools.
- **Week 2 – Intermediate Python and Tools:** Dive into more advanced Python topics such as file I/O and object-oriented programming. You'll also learn to use essential developer tools: the command line for running programs, and Git & GitHub for version control and collaboration. A second mini-project will solidify these skills.
- **Week 3 – Introduction to Web Development with Flask:** Transition into web development by learning about HTTP, the request-response cycle, and building web applications with Flask. You'll create routes, work with templates (HTML/CSS), handle user input via forms, and possibly serve dynamic content. A Flask mini-project will help practice these concepts.
- **Week 4 – Capstone Project and Advanced Topics:** Integrate everything you've learned to build a capstone full-stack web application using Flask, including a backend database for data persistence. We will also cover best practices in code formatting (PEP8), basic testing, documentation, and finally how to deploy your web app to a live server.

Throughout the course, each day lists **learning objectives** and includes theoretical explanations followed by **guided exercises or lab activities**. At the end of each week, you'll find a **quiz** to review key concepts, as well as a **mini-project** that applies that week's lessons. The **capstone project** in Week 4 will be a culmination of the skills acquired – you will build and deploy a small full-stack application from scratch. Instructions, requirements, and evaluation criteria for projects are clearly provided to guide you step-by-step.

How to Use this Coursebook: You are encouraged to follow the course in order, day by day, as concepts build on each other. Read the explanations, run the example code, and **practice by typing out code yourself**. Use the hands-on labs as an opportunity to apply what you learned – the exercises are designed to reinforce the topics of the day. Don't be afraid to experiment with the code or even make mistakes; debugging is a valuable skill you will develop. Make sure to complete the quizzes and projects to test your understanding. If you have an instructor or peers, discuss the concepts and get feedback on your code. By the end of the bootcamp, you should feel comfortable writing Python programs, using Git/GitHub, and developing a simple web application with Flask.

Let's get started on your coding journey!

Week 1: Python Programming Fundamentals

Overview: Week 1 will establish a strong foundation in programming using Python. We'll cover basic syntax, how to write and run Python code, variables and data types, operators, control flow (conditionals and loops), functions, and basic data structures (lists and dictionaries). By the end of the week, you will build a small text-based program using these fundamentals. We will also introduce basic command-line usage for running Python programs. Each day includes hands-on practice to build your confidence.

Day 1: Introduction to Programming and Environment Setup

Learning Objectives:

- Understand what programming is and what Python can do.
- Set up your development environment (install Python and a code editor).
- Run a simple Python script using the command line.
- Write your first Python program ("Hello, World!") and use the print function.
- Learn about basic data types (numbers, strings) and simple arithmetic operations in Python.

Theoretical Concepts:

On Day 1, we begin with the absolute basics. We'll discuss **what a programming language is** and why Python is a popular choice for beginners. Python is known for its simple, readable syntax and versatility (used in web development, data science, automation, and more) ¹ ² . We will ensure your computer is ready for coding by **installing Python 3** (if not already installed) and a code editor or IDE (Integrated Development Environment). Popular choices are Visual Studio Code, PyCharm, or even a simple text editor. This course will assume you're using VS Code or a similar editor that allows running Python easily. We'll also introduce the **command-line interface (CLI)** – a text-based way to interact with your computer. On Windows, this is the Command Prompt or PowerShell; on macOS/Linux, the Terminal. You will learn basic CLI commands like changing directories (`cd`), listing files (`ls` or `dir`), and running Python scripts. Using the CLI is important for running Python and later using Git.

Writing Your First Program:

Every programmer's starting point is the "Hello, World!" program. Open your editor and create a new file called `hello.py`. In it, type:

```
print("Hello, World!")
```

Save the file, then run it using the command line:

- In the terminal, navigate to the folder where `hello.py` is saved.
- Run the script by typing: `python hello.py` (the exact command may be `python3 hello.py` on some systems).

You should see the output: `Hello, World!` printed to the console. Congratulations – you just wrote your first Python program!

Next, we explore **basic data types** in Python: - **Integers** (int) – whole numbers (e.g., 1, 42, -5). - **Floats** (float) – decimal numbers (e.g., 3.14, 0.5). - **Strings** (str) – text enclosed in quotes (e.g., "hello", 'Python'). - **Booleans** (bool) – logical True/False values (we will encounter these more in Day 3).

We learn how to use Python like a calculator. In a Python shell or script, you can perform arithmetic:

```
# Basic arithmetic operations
print(5 + 3)      # Addition -> outputs 8
print(10 - 4)     # Subtraction -> outputs 6
print(2 * 3)      # Multiplication -> outputs 6
print(8 / 2)      # Division -> outputs 4.0 (result is float)
print(5 // 2)     # Floor division -> outputs 2 (integer division)
print(5 % 2)      # Modulus -> outputs 1 (remainder of division)
print(2 ** 3)     # Exponentiation -> outputs 8 (2 to the power of 3)
```

We also demonstrate working with strings:

```
print("Bootcamp " + "Day 1")  # String concatenation -> "Bootcamp Day 1"
print("Ha" * 3)               # String repetition -> "HaHaHa"
```

You will see how the `print()` function can display results of calculations or text. We explain that Python will automatically handle types for you (e.g., int vs float), but you cannot mix incompatible types without conversion (for instance, trying to add a number and a string will cause an error).

Hands-On Lab – Using the Python Interpreter:

Students should try using the Python **interactive interpreter (REPL)** to do simple calculations and print messages. Open a terminal and type `python` (or `python3`) to enter Python's interactive mode (`>>>` prompt). Then practice entering expressions like `2+2` or `print("Your Name")` and see the immediate results. This is a quick way to test ideas.

Guided Exercise – Simple Greeter:

Write a Python script `greet.py` that asks the user for their name and then greets them. Use the `input()` function to read user input from the console. For example:

```
name = input("What is your name? ")
print("Hello, " + name + "! Nice to meet you.")
```

Run the program and test it with different names. This exercise introduces string input/output and string concatenation. (We will discuss in Day 2 how `input()` always returns a string and how to convert input to numbers for calculations.)

By the end of Day 1, you should be comfortable creating and running simple Python scripts, and you've learned about basic data types and operations. You've also had a brief introduction to the command line for running Python. **Tip:** At this stage, focus on writing code and running it to see what happens. Don't worry if you encounter errors – they are part of the learning process! We will learn how to interpret error messages as we go along.

Day 2: Variables, Data Types, and Working with Strings

Learning Objectives:

- Understand what variables are and how to use them to store data.
- Learn about data types in more depth (integers, floats, strings, booleans).
- Perform type conversion (casting) between basic types.
- Use string methods for basic text manipulation.
- Practice reading input from the user and processing it.
- Learn best practices for writing clear code (meaningful variable names, simple commenting).

Theoretical Concepts:

On Day 2, we build upon the basics by introducing **variables** – named references to data. A variable allows you to store a value and use it later in your program. For example:

```
age = 15          # assign an integer to a variable
pi = 3.14159      # assign a float
message = "Good Morning"
```

We explain that Python uses the **assignment operator** `=` to assign values to variables. The variable name goes on the left, the value (or expression) on the right. Variables can be used in expressions after they are assigned:

```
x = 5
y = 2 * x + 3     # y will be 2*5 + 3 = 13
print(y)         # outputs 13
```

Important rules are covered: variable names must start with a letter or underscore, and can contain letters, digits, underscores (no spaces or special characters). They are case-sensitive (e.g., `Name` and `name` are different).

We also discuss data types more formally: - **Numbers:** integers vs floats. We highlight how division of two ints yields a float in Python (e.g., `5/2 = 2.5`), and mention that you can use `//` for integer division and `%` for remainder. - **Boolean:** a bool can be `True` or `False`. We will use them more when we cover conditions, but introduce the idea that comparison operators produce booleans (e.g., `5 > 3` is True). - **Type conversion:** sometimes you need to convert between types. For example, using `int()` to convert a string containing digits to an integer, or `str()` to convert a number to a string. We demonstrate:

```
age_str = "30"
age_num = int(age_str)      # convert "30" to 30 (int)
price = 9.99
price_str = str(price)     # convert 9.99 (float) to "9.99" (string)
```

- If the string is not numeric (e.g., `"hello"`), converting to int or float will cause an error.

Working with Strings:

Python provides many **string methods** that allow you to manipulate text. We show a few useful ones:

```
s = "Hello, World"
print(s.lower())    # "hello, world" (make lowercase)
print(s.upper())    # "HELLO, WORLD"
print(s.replace("World", "Python")) # "Hello, Python"
print(len(s))       # 12 (length of the string)
```

We also introduce **string formatting** as a better way to build strings that include variables. There are a few methods (f-strings, `format()` method, and %-formatting). We will use f-strings (available in Python 3.6+ for simplicity):

```
name = "Alice"
age = 20
print(f"My name is {name} and I am {age} years old.")
```

This will output: "My name is Alice and I am 20 years old." The f-string allows embedding variable values inside `{}` in the string.

Reading User Input:

We practiced `input()` in Day 1; today we'll do more. Remember `input()` returns a string, so if you need a number, convert it:

```
num_str = input("Enter a number: ")
num = int(num_str) # assuming the user enters a valid integer
print(f"{num} squared is {num**2}")
```

We emphasize **error handling** lightly here: if a user enters something that isn't a number, the above conversion will fail. We won't dive deep into error handling until later, but it's good to be aware that not all input is "well-formed".

Best Practices (Style) Note: Even as a beginner, it's good to start following basic code style guidelines. For example: - Use **meaningful variable names** that describe the data (e.g., `temperature_celsius` instead of `t`). - Use **lowercase_with_underscores** for variable and function names (this is Python's recommended style, often called "snake_case") ³. - Keep your code tidy with proper **indentation** (Python uses indentation to define code blocks). Use 4 spaces for each indent level ⁴ ⁵ (your editor can help by inserting spaces when you press the Tab key). Consistent indentation is critical in Python – misaligned indents will cause errors. - We'll discuss more style tips (PEP 8, the official Python style guide) in a later session, but it's worth starting to think about code readability now.

Hands-On Lab – Temperature Converter:

Create a program `temp_converter.py` that asks the user to input a temperature in Celsius, converts it to Fahrenheit, and prints the result. The formula is: $F = C * 9/5 + 32$. Steps to guide you: 1. Use `input()` to get a string from the user (e.g., "Enter temperature in Celsius: "). 2. Convert that string to a float (since temperatures can have decimals). 3. Apply the conversion formula to

compute Fahrenheit. 4. Use an f-string to print the result in a nice format, e.g., "25°C is 77°F." (Hint: You can include the degree symbol ° in strings).

Test your program with a few values (0°C should be 32°F, 100°C should be 212°F, etc.).

Guided Exercise – String Fun:

Practice string operations by writing a short script `string_fun.py` that:

- Takes a string input from the user (e.g., a phrase or word).
- Prints the string in reverse order. (Hint: Strings can be sliced with syntax `string[::-1]` to reverse, or you can use a loop to build the reversed string).
- Prints the string in all uppercase and all lowercase.
- (Challenge) Prints whether the string is a palindrome (reads the same forwards and backwards), ignoring case and spaces. *This challenge is optional.*

This exercise reinforces string indexing/slicing and methods. It's okay if you find the palindrome check tricky – we will cover loops tomorrow which can also be used for this.

By the end of Day 2, you should have a solid grasp of using variables to store data, manipulating strings, and converting between basic types. You have written small interactive programs that respond to user input. These are the building blocks for more complex logic coming next. Keep practicing writing code, and remember to run your programs frequently to see if they work as expected. Small, incremental testing is key to catching mistakes early.

Day 3: Control Flow – Conditionals and Loops

Learning Objectives:

- Learn how to make decisions in code using **if**, **elif**, **else** statements.
- Understand comparison and logical operators for building conditions.
- Use loops (**for** and **while**) to repeat actions.
- Learn how to loop through sequences (like ranges of numbers or characters in a string).
- Practice using control flow to solve simple problems (e.g. find even/odd, summing numbers).
- Gain familiarity with the concept of algorithmic thinking – breaking down tasks into step-by-step logic.

Theoretical Concepts:

Day 3 introduces the concept of **control flow**, which allows programs to execute certain code blocks conditionally or repeatedly, instead of running line-by-line from top to bottom only. This gives our programs “brains” – the ability to make decisions and perform tasks multiple times.

Conditionals (If Statements):

We use `if` statements to execute code only if a condition is True. Syntax:

```
if condition:
    # code block to execute if condition is True
elif another_condition:
    # code if the first condition was False, but this one is True
else:
    # code if none of the above conditions were True
```

We explain **comparison operators** used in conditions: `==` (equals), `!=` (not equals), `>`, `<`, `>=`, `<=`. We also cover **logical operators**: `and`, `or`, `not` for combining conditions. For example:

```
x = 10
if x > 0 and x % 2 == 0:
    print("x is a positive even number.")
elif x > 0 and x % 2 != 0:
    print("x is a positive odd number.")
else:
    print("x is zero or negative.")
```

In this snippet, the program prints a different message depending on the value of `x`. We highlight the importance of indentation – the indented block under an `if` (or `elif` / `else`) runs only when that condition is True.

We also point out that Python considers certain values as “Falsey” or “Truthy” in conditions. For instance, `0`, `""` (empty string), `None`, and empty collections (list, dict) are treated as False in an `if` condition context, whereas non-zero numbers, non-empty strings, etc., are True. However, to avoid confusion for beginners, we encourage using explicit comparisons (e.g., `if len(name) == 0:` instead of just `if name == "":`) until they are comfortable.

Loops:

Loops allow us to repeat code. There are two main loop types in Python: - `for` **loops** – used to iterate over a sequence (like a list, range of numbers, characters in a string, etc.). - `while` **loops** – repeat as long as a condition remains True.

A `for` loop example using Python’s built-in `range()` function to iterate over a range of numbers:

```
# Print numbers 1 to 5
for i in range(1, 6):
    print(i)
```

This will print 1 2 3 4 5 (each on a new line). We explain that `range(a, b)` generates numbers from a (inclusive) to b (exclusive). If omitted, it starts at 0 by default. So `range(5)` would produce 0,1,2,3,4. We also mention `range(start, stop, step)` for completeness, so they know it exists (e.g., `range(0, 10, 2)` gives even numbers 0,2,4,6,8).

We show iterating over a list:

```
fruits = ["apple", "banana", "cherry"]
for f in fruits:
    print(f)
```

This loop will print each fruit name. The loop variable `f` takes each value in the list in turn.

A `while` loop example:

```
count = 3
while count > 0:
    print("Countdown:", count)
    count -= 1    # decrement count by 1 each loop iteration
print("Blast off!")
```

This will print "Countdown: 3", "Countdown: 2", "Countdown: 1", then "Blast off!". We caution that `while` loops need careful handling to ensure the condition eventually becomes False, otherwise you create an infinite loop. For instance, if we forget `count -= 1`, the loop would never end because `count` would always be `> 0`. If you ever accidentally create an infinite loop during practice, you can stop your program (often by pressing **Ctrl+C** in the terminal).

We also introduce the `break` and `continue` statements: - `break` will exit the loop immediately. - `continue` will skip to the next iteration of the loop (jump back to the loop header, evaluating the condition in a while, or moving to the next item in a for).

For example:

```
for num in range(1, 10):
    if num == 5:
        break        # exit loop when num is 5
    if num % 2 == 0:
        continue     # skip even numbers
    print(num)        # this will print only odd numbers 1,3 then loop breaks
                    at 5
```

This snippet would output 1, 3 and then stop (because at num=5, we break out).

Hands-On Lab – Even or Odd:

Write a script `even_odd.py` that asks the user for an integer and tells them whether it's even or odd. Use an `if` statement and the modulus operator `%` (recall: `x % 2 == 0` means x is divisible by 2). Extend this by also checking if the number is zero or negative and handle those cases with additional conditions or an `elif` chain. Test your program with various inputs (negative, zero, positive even, positive odd).

Guided Exercise – FizzBuzz:

"FizzBuzz" is a classic programming challenge. Print the numbers from 1 to 20, but for multiples of 3 print "Fizz" instead of the number, for multiples of 5 print "Buzz", and for multiples of both 3 and 5 print "FizzBuzz". This exercise uses both loops and conditionals:

```
for n in range(1, 21):
    if n % 15 == 0:        # divisible by both 3 and 5
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
```



```
else:
    print(n)
```

Try to implement this on your own first, then compare with the above solution. This will test your understanding of conditionals order (the order of checks matters) and loop usage. It's also a fun problem that combines multiple concepts.

Guided Exercise – Summation with Loops:

Write a program `sum_numbers.py` that asks the user to enter a positive integer `N` and then calculates the sum of all numbers from 1 to N. For example, if N=5, the program should output 15 (since $1+2+3+4+5 = 15$). You can do this using a `for` loop:

```
total = 0
N = int(input("Enter a positive integer: "))
for i in range(1, N+1):
    total += i    # same as total = total + i
print(f"The sum of numbers 1 to {N} is {total}.")
```

Test this with different values of N. (We will later see there's a mathematical formula for this, but using a loop is a good practice.)

By the end of Day 3, you can make your programs *react* to different conditions and repeat tasks. These are powerful constructs: any algorithm can be constructed using sequence (Day 1-2 concepts), conditionals, and loops. You've practiced some classic problems like FizzBuzz which sharpen your logic. Continue to experiment – for example, try writing a program to find prime numbers or to print a simple pattern of stars (`*`). If you get stuck or encounter logic errors (bugs), don't worry – debugging is how you learn. Use print statements inside loops to trace what your code is doing if needed.

Day 4: Functions and Code Organization

Learning Objectives:

- Understand what **functions** are and why they are used (to reuse code and organize logic).
- Learn how to define functions in Python using `def` and how to call (invoke) them.
- Grasp the concept of parameters (inputs) and return values (outputs) of functions.
- Write some simple functions to perform tasks, and see how to use them in a program.
- Learn about the scope of variables (local vs global).
- (If time permits) Introduce **modules** and how to organize code into multiple files, and using Python's standard library for common tasks.

Theoretical Concepts:

At this point, your code in earlier days might be getting longer. Day 4 introduces **functions**, which are a way to package code into reusable blocks. A function is like a mini-program that can take input, do some processing, and optionally return an output. Functions help avoid repetition and make code more readable by giving a name to a set of actions.

Defining a Function:

In Python, we define a function using the `def` keyword:

```
def greet_user(name):
    """Greets the user by name.""" # This is a docstring describing the
    function
    print(f"Hello, {name}!")
```

This defines a function named `greet_user` that takes one parameter `name`. When called, it prints a greeting. The triple-quoted string right after the function header is a **docstring**, which is a documentation comment explaining what the function does (this is a good habit for writing clear code). To use (call) this function, we simply do:

```
greet_user("Alice") # Output: Hello, Alice!
greet_user("Bob")   # Output: Hello, Bob!
```

We explain that `name` is a **parameter** (a variable that gets its value when the function is called). "Alice" and "Bob" here are **arguments** passed to the function. Functions can have multiple parameters or none at all.

Return Values:

By default, a function returns `None` if it doesn't hit a `return` statement. Often we want a function to compute something and give back a result. For example:

```
def add(a, b):
    """Returns the sum of a and b."""
    return a + b

result = add(3, 4) # result will be 7
print(result)
```

Here, the function `add` takes two numbers and returns their sum. The `return` statement exits the function and sends the value back to the caller. We emphasize that once `return` is executed, the function ends – code after `return` in the function body won't run.

We also cover a function with no parameters:

```
def say_hello():
    print("Hello!")
```

And one with multiple parameters and maybe more complex logic:

```
def is_even(n):
    """Checks if a number is even. Returns True if even, False otherwise."""
    if n % 2 == 0:
        return True
    else:
        return False
```

We can shorten that function by returning the boolean expression directly (`return n % 2 == 0`), but we show the longer form for clarity.

Variable Scope:

We introduce the concept of **local variables** (variables defined inside a function) versus **global variables** (defined at the top level of a script). A variable inside a function (like `a` and `b` in `add`, or `n` in `is_even`) is local to that function – it doesn't exist outside the function. For example:

```
def f():
    x = 5    # x is local to f
    print(x)

f()
print(x)    # This will cause an error because x is not defined outside f
```

This concept of scope is important to avoid confusion when writing larger programs. We generally use function parameters and return values to get data in and out of functions, rather than relying on global variables. Global variables (declared at top of file) can be accessed inside functions, but to modify them you'd need to declare them `global` within the function (we rarely recommend this for beginners; instead design functions to return values that you then assign to globals if needed).

Organizing Code into Multiple Functions:

We demonstrate how a program can call multiple functions and how functions can call other functions. For instance, building on previous days:

```
def calculate_factorial(n):
    """Calculate factorial of n (n!)."""
    result = 1
    for i in range(1, n+1):
        result *= i
    return result

def main():
    num = int(input("Enter a positive integer: "))
    fact = calculate_factorial(num)
    print(f"The factorial of {num} is {fact}")

# Call the main function to run the program
main()
```

Here `main()` function drives the program using a helper function `calculate_factorial`. This separation of concerns is useful: `calculate_factorial` knows how to compute a factorial, and `main` handles input/output. This makes testing easier too (we could test `calculate_factorial` independently of user input).

Modules (Optional Intro):

If time permits, mention that as programs grow, you can organize functions into different files called

modules. For example, you might have `math_utils.py` containing math-related functions, and then import those in your main program:

```
import math_utils
result = math_utils.calculate_factorial(5)
```

or use `from math_utils import calculate_factorial`. We won't dive deep today, but it's good to know that Python's standard library is full of modules you can import to use pre-written functions (e.g., `math`, `random`, etc.). For instance:

```
import math
print(math.sqrt(16)) # uses the sqrt function from math module
```

This prints 4.0. Or:

```
import random
print(random.randint(1, 6)) # simulates a dice roll (random int between 1
and 6 inclusive)
```

We encourage experimenting with these standard modules.

Hands-On Lab – Simple Calculator Function:

Create a file `calculator.py`. Inside it, define four functions: `add(x, y)`, `subtract(x, y)`, `multiply(x, y)`, `divide(x, y)` – each performing the basic arithmetic and returning the result. Also include a `main()` function that: 1. Prompts the user to enter two numbers (you can use `float(input("..."))` to allow decimals). 2. Prompts the user to choose an operation (e.g., `+` `-` `*` `/` or even words "add", "subtract"...). 3. Calls the appropriate function based on the choice. 4. Displays the result.

Use `if/elif` to decide which function to call. For example:

```
if op == '+':
    result = add(a, b)
elif op == '-':
    result = subtract(a, b)
# ... and so on
```

Make sure to handle the division case carefully (what if the user tries to divide by zero? You can check and print a warning instead of calling `divide` in that case). This exercise ties together conditionals, user input, and function calls.

Guided Exercise – Palindrome Function:

Take your palindrome-checking logic from Day 2's string exercise (if you did it), and turn it into a function `is_palindrome(text)` that returns `True` if `text` is a palindrome, `False` otherwise. Remember to ignore case and spaces. You might use `text.lower()` to normalize case and perhaps `replace(" ", "")` to remove spaces before checking. Test this function with a few known palindromes (e.g., "racecar", "Never odd or even", "hello").

Then integrate this into a simple program: prompt the user for a phrase and use the function to tell them if it's a palindrome. This demonstrates how functions make your code modular and reusable (you could reuse `is_palindrome` elsewhere easily).

By the end of Day 4, you have learned to structure your code better through functions. This will be crucial as we build larger programs. Functions help break down complex tasks into manageable pieces. You also got a taste of Python's vast library of modules. Tomorrow, we'll introduce some basic data structures and finalize the Week 1 project that will incorporate many concepts learned so far.

Day 5: Data Structures (Lists & Dictionaries) and Mini-Project 1

Learning Objectives:

- Learn about **lists** in Python: creating lists, accessing and modifying elements, common list methods.
- Understand **dictionaries** in Python: storing key-value pairs, and basic operations on dicts.
- (Briefly) mention other data structures (tuples, sets) for awareness.
- Work on a **mini-project** that incorporates Week 1 concepts (variables, loops, conditionals, functions, and data structures).
- Practice using Git and GitHub to save your project code (introduction to version control).

Theoretical Concepts:

On Day 5, we introduce fundamental **data structures** which allow you to store collections of data. Python's most versatile data type is the **list**. A list is an ordered collection of elements (which can be of any type, and the list can mix types, though usually they're homogeneous). Lists are defined with square brackets `[]`. For example:

```
numbers = [10, 20, 30, 40]
words = ["hello", "world"]
mixed = [1, "two", 3.0]
```

We explain that list elements are **indexed** starting at 0. So in `numbers`, `numbers[0]` is 10, `numbers[1]` is 20, etc. Negative indices access from the end (`numbers[-1]` would be 40). We demonstrate how to **slice** lists to get sublists:

```
print(numbers[1:3]) # [20, 30] (slice from index 1 up to but not including 3)
```

Common list operations: - **Append**: `numbers.append(50)` adds 50 to the end of the list. - **Remove**: `numbers.remove(20)` removes the first occurrence of 20. - **Pop**: `numbers.pop()` removes the last element and returns it; `numbers.pop(0)` removes the first element. - **Length**: `len(numbers)` gives the number of elements. - **Iteration**: you can loop through lists as shown on Day 3. Also, `for i, val in enumerate(numbers):` gives you both index and value.

Next, we cover **dictionaries (dict)** – another powerful built-in data structure for storing data as key-value pairs (like a real-world dictionary word -> definition). Syntax uses curly braces `{}`:

```
capitals = {"France": "Paris", "Japan": "Tokyo", "India": "New Delhi"}
```

Here "France" is a key and "Paris" is its value, etc. We explain: - Keys must be unique and an immutable type (e.g., strings, numbers, etc., but not lists). - Values can be any type. - To access a value: `capitals["France"]` yields "Paris". If a key isn't present, you get a `KeyError` (we might mention the `get()` method to provide a default: `capitals.get("Germany", "Not found")`). - Adding/ updating an entry: `capitals["Germany"] = "Berlin"` would add Germany with Berlin, or update if "Germany" was already a key. - Removing an entry: `del capitals["India"]` removes that pair. - Looping: You can loop through dictionaries: `for country, city in capitals.items(): print(country, "->", city)`.

We might mention other data types briefly: - **Tuple:** like a list but immutable (cannot change elements once created). Written with parentheses, e.g., `point = (4, 5)`. Tuples often used for fixed collections of items. - **Set:** an unordered collection of unique elements. E.g., `unique_numbers = {1, 2, 5}`. Useful for membership tests and eliminating duplicates.

Given time constraints, we won't go deep into sets/tuples, but let students know they exist.

Mini-Project 1: "Guess the Number" Game (Console Application)

For the end of Week 1, we will build a small interactive game combining everything learned. The project is a **"Guess the Number"** game: - The program will randomly choose a secret number, and the user has to guess it. - After each guess, the program informs the user if the guess is too high, too low, or correct. - We'll use a loop to allow multiple guesses until the user finds the number or chooses to quit. - We will utilize functions to organize the code, and use conditional logic for comparisons. - We'll also use the `random` module to generate the secret number.

Step-by-Step Project Instructions:

1. **Setup:** Create a new Python file called `guess_number.py`. At the top, import the `random` module:

```
import random
```

We'll use `random.randint(1, 100)` to generate a random number between 1 and 100.

2. **Plan the Logic:** Decide on how the game will flow:
3. The computer picks a secret number.
4. Prompt the player to enter a guess.
5. If the guess is higher than the secret, print "Too high, try again."
6. If the guess is lower, print "Too low, try again."
7. If the guess is correct, congratulate the player and end the game.
8. Optionally, count the number of guesses and display it at the end.
9. Also allow the player to exit early by maybe typing a special value (like 0 or "exit").

10. Implement Step by Step:

11. Generate the secret number: `secret = random.randint(1, 100)`.
12. Initialize a guess counter to 0.
13. Use a `while True` loop to repeatedly ask for input:

```
while True:
    guess_str = input("Guess the number (1-100) or type 'quit' to exit: ")
    if guess_str.lower() == 'quit':
        print("Game exited. The number was", secret)
        break
    try:
        guess = int(guess_str)
    except ValueError:
        print("Please enter a valid number.")
        continue
    # increment guess counter
    # compare guess to secret and respond accordingly
```

Here we used a `try/except` to catch non-integer inputs (like letters) and prompt again. This is a gentle introduction to error handling; it's okay if you haven't seen `try/except` before – just provided as a template.

14. Inside the loop, after converting to int, compare the guess:

```
guesses += 1
if guess < secret:
    print("Too low! Try again.")
elif guess > secret:
    print("Too high! Try again.")
else:
    print(f"Congratulations! You guessed it in {guesses} tries.")
    break
```

15. This loop continues until the user either breaks out by guessing correctly or typing 'quit'.
16. **Test the Game:** Run your program and play a round. Does it behave as expected for various scenarios (too high, too low, quitting, non-number input)? Adjust as needed.
17. **Enhancements (Optional):** If you finish quickly, you can consider enhancements:
18. Limit the number of guesses and end the game if exceeded, offering to reveal the number or let the user try again.
19. After a correct guess, ask if the user wants to play again (loop back to start with a new secret number).
20. Provide hints after a certain number of guesses (e.g., "Hint: the number is divisible by 3").

This mini-project is a great way to wrap up Week 1. It uses random number generation, loops, conditionals, user input, conversion, and can be broken into functions (for instance, you could put the main game loop in a function `play_game()` to practice function use).

Using Git and GitHub – Version Control Introduction:

Since this is the end of the week, it's a perfect time to introduce **Git**, a version control system, and **GitHub** for remote code storage and collaboration. Version control is essential for developers to track changes and collaborate with others without overwriting each other's work. **Git** helps you save and manage different versions of your files and code, and work with others while keeping track of changes ⁶. We will keep it simple today: create a local repository for your Week 1 project and push it to GitHub.

Setting up Git (if not already done): - Install Git on your system (if not pre-installed). On Windows, use the Git installer; on macOS, Git might be present or installed via Xcode Command Line Tools; on Linux, install via package manager. - Configure your Git username and email (this is one-time setup):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

These identify your commits.

Initializing a Repository: 1. In your project directory (the folder containing `guess_number.py` and other files from Week1), open the terminal. 2. Run `git init`. This creates a new Git repository in that folder (a hidden `.git` directory will be created). 3. Run `git status` to see the status. Initially, your files are untracked. 4. Run `git add .` to stage all files (or `git add filename` for specific files). 5. Run `git commit -m "Initial commit - add guess_number project"`. This commits the files to your repository with a message.

Now you have a local version history. If you edit a file, you would `git add` and `git commit` again with a message describing the change (e.g., "fix loop condition in guess game").

Pushing to GitHub: - Create a new repository on GitHub via the web interface (give it a name, e.g., `bootcamp-projects`). **Do not initialize with a README** (because you already have a commit locally). - In your local repo, link it to GitHub:

```
git remote add origin https://github.com/YourUsername/bootcamp-projects.git
git branch -M main
# rename the local default branch to 'main' if it's not already
git push -u origin main # push your commits to GitHub
```

- Refresh your GitHub repo page; you should see your code now uploaded.

From now on, you can commit changes locally and push (`git push origin main`) to update the remote. If working on multiple computers, you'd `git pull` to fetch the latest code. Using Git will become more crucial as projects grow, and we will continue practicing it in subsequent weeks (for example, you will version control your Flask app code as you develop it).

End-of-Week 1 Quiz: (Multiple choice / short answer)

1. What is the correct syntax to start a function in Python?

A. `function myFunc():`

- B. `def myFunc():`
- C. `begin myFunc()`
- D. `create function myFunc:`

2. If `x = 5`, what will the following code print?

```
if x % 2 == 0:  
    print("even")  
else:  
    print("odd")
```

- A. `even`
- B. `odd`
- C. It will cause an error.
- D. Nothing will print.

3. What is the output of the following code?

```
fruits = ["apple", "banana", "cherry"]  
print(fruits[1])
```

- A. `apple`
- B. `banana`
- C. `cherry`
- D. This code has an error (list indices start at 1, not 0).

4. Which of these is NOT a valid variable name in Python?

- A. `total_sum`
- B. `value2`
- C. `3rd_value`
- D. `user_name`

5. **Short Answer:** What does the `append()` list method do? Give an example of how to use it.

6. **Short Answer:** Explain the difference between `=` (assignment) and `==` (equality operator) in Python.

7. **Short Answer:** How do you handle situations where a user's input might cause a program to throw an error, for example if you expect an integer but the user types letters?

(End of Week 1 content. Proceed to Week 2...)

Week 2: Intermediate Python and Developer Tools

Overview: In Week 2, we will expand our Python knowledge and introduce essential tools and practices for software development. We'll cover topics like file handling (reading from and writing to files), error

and exception handling, and the basics of object-oriented programming (classes and objects) to organize complex data. Additionally, a key focus this week is on using **Git and GitHub** more deeply and mastering the **command line** for various tasks. By mid-week, we'll start bridging towards web development by discussing how Python can interact with the internet and other systems (for instance, making a simple HTTP request or preparing to set up a web framework environment). The week's project will likely be a console application that incorporates file I/O and possibly a simple class to manage data (for example, a contact book). We'll end the week with another quiz and mini-project. Let's dive in!

Day 1: CLI Mastery and Version Control with Git/GitHub

Learning Objectives:

- Improve proficiency with **Command Line Interface (CLI)** commands for navigation and file management.
- Initialize a new Git repository and understand the workflow of `git add`, `git commit`, `git push`, etc.
- Understand the concept of commit history and how to inspect it (`git log`).
- Learn to create a repository on GitHub and push local commits to the remote repo.
- Practice collaboration basics: making and merging changes (intro to `git pull` and resolving simple merge scenarios).
- Use version control on an existing project (for example, start tracking the code from Week 1 if not already done).

CLI (Command Line) Review and Tips:

We briefly review crucial CLI commands in more depth: - `pwd` (print working directory) to know where you are. - `ls` (or `dir` on Windows) to list files in the current directory. - `cd foldername` to change directory; `cd ..` to go up one level. - `mkdir project` to create a new directory. - `touch file.txt` or using editors to create a file (Windows users can use `type NUL > file.txt` as an equivalent). - `rm` (or `del` on Windows) to remove files, `rmdir` to remove directories (with caution). - Using arrow keys to navigate command history, and tab for auto-completion.

These skills make you efficient in setting up projects and navigating to run scripts or manage files. We encourage you to use the CLI to run Python programs (as we did in Week 1) and now to use Git.

Git Basics Recap/Introduction:

We introduced Git at the end of Week 1. Today we'll reinforce and expand that knowledge. Git is a **distributed version control system** – it keeps track of changes to your files in a timeline of snapshots called commits. Each commit has a message and an ID. GitHub (or other platforms like GitLab, Bitbucket) is a remote hosting service for your Git repositories, enabling collaboration.

Key Git concepts: - **Repository (repo)**: A project folder tracked by Git. - **Staging area**: Where you place changes (`git add`) that you want to commit. - **Commit**: A snapshot of the staged changes with a message. - **Branch**: A pointer to a series of commits. By default, we use the **main** branch. Branches allow parallel development (more on this later, but we'll mostly work on main/master for now). - **Remote**: A copy of your repo on a server (like GitHub). Typically named `origin` by default when linking your local repo to remote.

Common Git Commands (Workflow): 1. `git init` – initialize a new repo (we did this already for our Week1 project). 2. `git status` – check status of changes. 3. `git add <file>` – stage a specific file

(or use `git add .` to stage all changes in current directory). 4. `git commit -m "message"` - commit the staged changes with a descriptive message. 5. `git log` - view commit history. 6. `git remote add origin <repo_url>` - connect your local repo to a remote (done once). 7. `git push -u origin main` - push commits to the remote repository (the `-u` sets the default upstream so future `git push` can be run without specifying branch). 8. `git pull` - fetch and merge changes from the remote to your local (important when collaborating or switching computers).

We will walk through an example. Let's say we have the Week1 project in a folder `bootcamp-week1` which we initialized with Git and maybe already pushed. Today, create a new repository for Week2 exercises (or continue in the same repo in a subfolder): - Create a directory `bootcamp-week2`, move into it in the terminal. - Run `git init` to start version control. - Perhaps create a simple file to test (e.g., `day1.txt` or a Python file) and then `git add` and `git commit`.

If you already have a GitHub repo for all bootcamp code, you can just add/commit within that. Or create a new repo on GitHub for week2. The instructor can guide on a strategy, but key is to practice the commands.

Collaboration scenario: (even if you're not collaborating in this course, understanding this is good) - If working with a partner on a project, you both clone the same GitHub repo. - One person's changes must be pulled by the other to integrate. For example, if you committed and pushed, your partner would run `git pull origin main` to get the updates. - If both modify the same file in overlapping ways, a **merge conflict** can occur, which Git will flag. You then have to edit the file to reconcile the differences and commit the resolved version. We won't delve deeply into conflicts now, but be aware of the need to coordinate and pull changes frequently.

Hands-On Lab – Git Practice:

We will create a small text file and practice version control on it: 1. In your week2 directory, create a file `story.txt`. Write one line: e.g., "Once upon a time...". 2. Initialize git (`git init` if not already done), add and commit this file (`git add story.txt`, `git commit -m "Add opening line of story"`). 3. Edit the file by adding another line of text. Then see `git diff` (it will show what changed). Stage and commit again (`git commit -am "Add second line"` using `-a` to skip separate add since file was already tracked). 4. Use `git log --oneline` to see a condensed history of your commits. 5. Create a GitHub repo (if not done) and push the commits (`git push`). Confirm on GitHub the file is updated. 6. (Optional) Now pretend another collaborator adds a line via GitHub web editor or another machine - for simulation, you can edit the file on GitHub's interface and commit. Then on your local, do `git pull` to retrieve that change.

Through this lab, you solidify the mental model of how changes move between your working directory, staging, repository, and remote.

By the end of Day 1 (Week2), you should be more comfortable using the command line and have a basic grasp of using Git for version control and GitHub for remote repositories. These skills will be invaluable as you start larger projects (like the Flask app soon) - you'll be able to save your work incrementally and collaborate or seek help by sharing your repository. Keep your Week1 project under version control as well if possible, and continue to commit changes as you improve it or fix bugs.

Day 2: File I/O and Exception Handling

Learning Objectives:

- Learn how to **read from and write to files** in Python (file I/O).
- Understand file paths and working with files on disk.
- Use Python's `with open(...)` context manager for safe file operations.
- Learn about **exceptions** (errors at runtime) and how to handle them using `try`, `except`, and `finally` blocks.
- See common exceptions (`ValueError`, `FileNotFoundError`, etc.) and how to avoid or handle them gracefully.
- Apply file I/O and exception handling in a mini-lab, e.g., reading a data file or writing logs.

File I/O (Input/Output):

Working with files is a common task – whether it's reading configuration, data, or writing output, logs, etc. Python makes file handling straightforward. The basic steps: 1. Open a file. 2. Read from or write to the file. 3. Close the file.

It's strongly recommended to use Python's **context manager** (`with` statement) to handle files, as it will automatically close the file for you, even if errors occur. Syntax:

```
with open("data.txt", "r") as file:    # "r" mode = read
    data = file.read()
    # do something with data
# file is now closed automatically
```

If you don't use `with`, you'd do:

```
file = open("data.txt", "r")
data = file.read()
file.close()
```

But forgetting to close can lead to resource leaks, so `with` is preferred.

We explain **file modes**: - `"r"` = read (file must exist). - `"w"` = write (creates file or truncates existing to zero length). - `"a"` = append (write without truncating, starts at end of file if exists). - `"r+"` = read and write.

Reading from a file:

Assume `data.txt` contains:

```
Alice,24
Bob,19
Charlie,30
```

Examples:

```
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

This reads the entire file into one string. If the file is large, this might be memory heavy; often we read line by line:

```
with open("data.txt", "r") as f:
    for line in f:
        line = line.strip() # remove trailing newline and spaces
        print("Line:", line)
```

This will iterate over each line in the file. Alternatively, `f.readlines()` returns a list of all lines.

Writing to a file:

```
with open("output.txt", "w") as f:
    f.write("Hello file!\n")
    f.write("This is a second line.\n")
```

This will create/overwrite `output.txt` with the given lines. We include `\n` for new lines because `write()` doesn't automatically add one (unlike `print` which adds newline by default). If you want to append:

```
with open("output.txt", "a") as f:
    f.write("Additional line.\n")
```

Now, understand that file paths can be relative or absolute. `"data.txt"` refers to a file in the current working directory. We show how to specify a path (on Windows use backslashes `\\` or raw string, on Linux/mac use forward slashes). For example: `open("C:\\Users\\me\\documents\\data.txt")` or `open("/home/me/data.txt")`. We won't go deep into path handling, but mention `os` module can join paths (`os.path.join`) to avoid manual separators, and that Python 3.4+ has a `pathlib` module with object-oriented path handling.

Exception Handling:

While running programs, things can go wrong (runtime errors). These generate **exceptions**. For example: - Trying to convert a non-numeric string to int raises `ValueError`. - Dividing by zero raises `ZeroDivisionError`. - Accessing a file that doesn't exist raises `FileNotFoundError`. - Many more (IndexError, KeyError, TypeError, etc.).

If not handled, exceptions will crash your program and print a traceback. In small scripts, that's fine; but in more robust applications, we handle exceptions to either recover or at least exit gracefully with a message.

Python's `try/except` syntax:

```
try:
    # code that might raise an exception
except SomeException as e:
    # code to handle that exception
```

Optionally:

```
else:
    # runs if no exception occurred in try
finally:
    # runs no matter what (after try/except blocks)
```

Example:

```
try:
    num = int(input("Enter an integer: "))
    print("100 / num =", 100/num)
except ValueError:
    print("That's not an integer! Please try again.")
except ZeroDivisionError:
    print("You entered 0, cannot divide by zero.")
```

In this case, if the input cannot be converted to int, it prints a friendly message instead of a stack trace. If 0 is entered, it catches that specific error. If any other exception occurred, it's not caught here and would still crash (we could use a broad `except Exception:` to catch anything, but usually you catch specific exceptions you expect).

Finally is often used to ensure some cleanup happens (like closing a file or releasing resources) regardless of errors, but since we use `with` for files, that's handled.

Using exceptions for file handling:

If you try to open a non-existent file in read mode, a `FileNotFoundError` occurs. We can handle it:

```
filename = "nosuchfile.txt"
try:
    with open(filename, "r") as f:
        data = f.read()
except FileNotFoundError:
    print(f"Error: The file {filename} was not found.")
```

This way, the program doesn't crash; it gives a clear message and can continue. You could even prompt for a new filename in the exception handler, or create the file, depending on context.

Hands-On Lab – Reading and Writing Files:

We will create a small program that reads a list of names from a file, and writes a greeting for each into another file. 1. Make a file `names.txt` manually (or have the program prompt you to enter names and write to the file). Put several first names, one per line. 2. Write a Python script `greet_names.py` that

attempts to open `names.txt` for reading. Use a try/except to handle if the file isn't found (perhaps prompt the user to create the file or ensure it's in correct location). 3. Read each name, strip newline, and for each name, write a line in an output file `greetings.txt` like: "Hello, \<Name>!\n".

```
try:
    with open("names.txt", "r") as infile, open("greetings.txt", "w") as
outfile:
    for line in infile:
        name = line.strip()
        greeting = f"Hello, {name}!\n"
        outfile.write(greeting)
except FileNotFoundError:
    print("names.txt not found. Please make sure it exists.")
```

Note: We used a neat Python feature: opening two files in one `with` statement by separating with a comma. 4. After running, open `greetings.txt` (in a text editor or via Python) to verify the content.

Guided Exercise – Simple Log Writer:

Imagine your program performs certain actions and you want to log messages to a file for debugging. Implement a function `log_message(message)` that opens an `app.log` file in append mode and writes the message with a timestamp. Use the `datetime` module for timestamp:

```
from datetime import datetime

def log_message(msg):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("app.log", "a") as log:
        log.write(f"[{timestamp}] {msg}\n")
```

Wrap the file operation in try/except if there could be issues (though appending to a file rarely fails except for permission issues or disk full). Then in a test part of your script, call this function a few times, e.g., `log_message("Program started")`, `log_message("Performed action X")`. Check `app.log` content after. This shows file writing and how logging might work.

By the end of Day 2, you should be comfortable with reading and writing files and know how to handle errors without crashing your program. You've also seen how exceptions work in general – an important part of robust programming. Continue to use try/except in moderation; catch exceptions you expect and can handle, and avoid silent pass statements that hide errors completely. Tomorrow, we'll move onto another big concept: object-oriented programming.

Day 3: Introduction to Object-Oriented Programming (OOP)

Learning Objectives:

- Understand the basics of **object-oriented programming** and why it's useful for structuring code.
- Learn how to define a **class** in Python to create objects (instances).
- Understand attributes (variables) and methods (functions) within classes.

- Learn about special method `__init__` for object initialization (constructors).
- Create and use objects; understand the `self` parameter.
- See a simple use-case for a class (e.g., modeling a real-world entity like a `Book`, `Student`, or in a project context, maybe a `Contact` in an address book).
- Briefly touch on concepts of encapsulation and maybe mention inheritance (though might leave inheritance/polymorphism for advanced courses).

What is OOP?

Object-Oriented Programming is a paradigm where we organize code into objects – each object is an instance of a **class** and bundles data (attributes) and behaviors (methods) that belong together. This can make managing complex programs easier by modeling concepts in code more naturally.

We've been using objects already (everything in Python is an object). For example, a string is an object of class `str` with methods like `.upper()`, `.find()`, etc. When you do `"hello".upper()`, you're calling a method of the string object.

Defining a Class:

In Python:

```
class Dog:
    def __init__(self, name, age):
        # initializer (constructor) method
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} says: Woof!")
```

Here we define a class `Dog` with an `__init__` method and a regular method `bark`. Points to explain: - `__init__` is a special method that runs when a new object is created. It is used to initialize the object's attributes. The parameter `self` refers to the instance itself (like "this" in some other languages), and we use it to set attribute values. When we create a `Dog`, we might do: `d = Dog("Rex", 5)` - Python will call `Dog.__init__(d, "Rex", 5)` internally to set up the object. - We attach attributes to `self`, e.g., `self.name = name` creates an attribute `name` for that instance. - The `bark` method uses `self.name` to access the dog's name and prints a message. `self` must be the first parameter of any instance method, and it is automatically provided when you call the method on an object (so `d.bark()` calls `Dog.bark(d)` behind the scenes).

Creating Objects (Instances):

Using the above class:

```
dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 2)
dog1.bark()    # "Buddy says: Woof!"
dog2.bark()    # "Max says: Woof!"
print(dog1.age) # 3
dog2.age += 1
print(f"{dog2.name} is now {dog2.age} years old.")
```


Each object `dog1` and `dog2` has its own `name` and `age` attributes. They both have a `bark` behavior defined by the class.

We explain that classes are like blueprints, and objects are like actual houses built from those blueprints. Classes define what attributes and methods objects will have.

Why use classes?

If we had to manage a lot of related data, classes help group them logically. For instance, if building a contact manager, instead of using parallel lists for names, phones, emails, we can define a `Contact` class with those attributes, and maybe methods to pretty-print contact info or update it. This makes code more intuitive and encapsulated.

Encapsulation:

This is the idea that an object controls its own state via its methods, and users of the object shouldn't need to directly manipulate its internal state. In Python, there's no strict access control like private/public as in some languages, but by convention a leading underscore `_` on a variable means "don't touch this outside the class".

Example Use-Case:

Let's walk through a simple use case – managing a collection of books in a library system.

```
class Book:
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = year
        self.checked_out = False

    def checkout(self):
        if self.checked_out:
            print(f"Sorry, '{self.title}' is already checked out.")
        else:
            self.checked_out = True
            print(f"You have checked out '{self.title}'. Please return it on time.")

    def return_book(self):
        if self.checked_out:
            self.checked_out = False
            print(f"Thank you for returning '{self.title}'.")
        else:
            print(f"'{self.title}' was not checked out.")
```

Now using it:

```
book1 = Book("1984", "George Orwell", 1949)
book2 = Book("The Hobbit", "J.R.R. Tolkien", 1937)
book1.checkout()    # You have checked out '1984'.
```

```
book1.checkout()    # Sorry, '1984' is already checked out.
book1.return_book() # Thank you for returning '1984'.
```

This demonstrates maintaining state (`checked_out` flag) and operations that modify it with logic, encapsulated in methods.

We likely won't delve into **inheritance** deeply (where one class can inherit attributes/methods from another), but we might briefly mention you could have e.g. a subclass `EBook` of `Book` that adds a file size attribute or something, just to hint at OOP's extensibility. But it's okay to skip that if it's too much.

Hands-On Lab – Creating a Class:

Let's implement a simple **Contact** class for an address book (as we foreshadowed):

```
class Contact:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
    def display(self):
        print(f"{self.name}: {self.phone}")
```

1. Create a few Contact instances in a script:

```
c1 = Contact("Alice", "123-4567")
c2 = Contact("Bob", "987-6543")
c1.display()    # Alice: 123-4567
c2.display()    # Bob: 987-6543
```

2. Perhaps store them in a list: `contacts = [c1, c2]`. Loop through and call `display()` on each.
3. Add a new attribute or method if you want. For instance, add `email` as optional (maybe give it a default of `None` if not provided). Or write a method `update_phone(new_phone)` that changes the phone number.

```
def update_phone(self, new_phone):
    self.phone = new_phone
```

Try calling `c1.update_phone("555-0000")` and then `c1.display()` to see the change.

Guided Exercise – Bank Account Class:

Design a class `BankAccount` with attributes `owner` (name of account holder) and `balance` (starting balance, default 0). Add methods: - `deposit(amount)` - adds to balance. - `withdraw(amount)` - subtracts if sufficient funds, otherwise print an error. Example implementation:

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
    def deposit(self, amount):
```

```

        self.balance += amount
        print(f"${amount} deposited. New balance: ${self.balance}")
    def withdraw(self, amount):
        if amount > self.balance:
            print(f"Insufficient funds for withdrawal of ${amount}. Current
balance: ${self.balance}")
        else:
            self.balance -= amount
            print(f"${amount} withdrawn. New balance: ${self.balance}")

```

Test it:

```

acct = BankAccount("John Doe", 100)
acct.deposit(50)    # should add 50
acct.withdraw(30)   # should succeed
acct.withdraw(150)  # should show insufficient funds

```

This class design practice reinforces understanding of methods and internal state.

By the end of Day 3, you should grasp how to model simple entities as classes in Python and create/use objects. While small scripts might not *require* classes, as projects grow (e.g., a web app), thinking in terms of objects can help manage complexity. In our upcoming Flask application, we might use classes for things like database models or form representations (Flask itself and its extensions often use OOP). So this knowledge will come in handy.

Day 4: Preparing for Web Development – Intro to Flask and Web Concepts

Learning Objectives:

- Understand the basics of how the web works: **clients** and **servers**, **HTTP** protocol, requests and responses.
- Learn what a web framework like Flask provides and why we use it instead of writing low-level network code.
- Set up a Python virtual environment and install Flask.
- Write a minimal Flask application ("Hello, World" web server).
- Understand the concept of routes/URLs and how to define view functions for different routes.
- Run the Flask development server and access the web page via browser.
- (If time) Introduce the idea of templates (HTML files) and returning HTML from Flask, to be expanded in Week 3.

Today we transition from standalone Python programs to web applications. This is a big shift: now our Python code will respond to **HTTP requests** from a web browser (or other client), and send back responses (like HTML pages).

How the Web Works (Crash Course):

A **client** (usually a web browser) sends an **HTTP request** to a server, which processes it and returns an **HTTP response**. The response typically includes a status code (200 for success, 404 for not found, etc.) and the content (e.g., an HTML page). The browser then renders the HTML for the user. We have a server (our Flask app) running that listens for requests on a certain URL (like `http://localhost:5000/`) and returns responses.

Simplified HTTP request-response cycle between a client (browser) and a server. The client sends a request (e.g., asking for a web page), the server processes it (perhaps interacting with a database or application logic), and then sends back an HTTP response (often HTML content) for the browser to display ⁷.

Key terms: - **HTTP Methods:** GET (for retrieving information), POST (for sending data like form submissions), etc. We'll mainly use GET for showing pages and POST for form submissions. - **URL/Route:** The path in the web address corresponds to a route on the server (e.g., `/about` or `/login`). - **Status Code:** Part of the HTTP response (200 OK, 404 Not Found, 500 Server Error, etc.).

We won't dive too deep into HTTP details right now, but this context is important so you understand what Flask is doing.

What is Flask?

Flask is a **micro web framework** for Python. "Micro" because it's lightweight and minimal, giving you the core tools to build web apps (routing, request/response handling, etc.) without much overhead. Despite being "micro," Flask is powerful and has many extensions for database integration, authentication, etc. It's designed to make getting started quick and easy ⁸.

In essence, Flask will: - Start a web server for you (for development). - Map URL routes to Python functions you write (these are called *view functions* or *route handlers*). - Handle details of HTTP, so in Python you can work with request data easily and return responses. - Provide a template engine (Jinja2) for generating HTML dynamically (we will use this in Week 3). - Remain flexible – you choose what components to add (like a database layer, etc.).

Setting up Flask: 1. **Virtual Environment:** It's a good practice to use a virtual environment for each project, to manage dependencies. Let's create one: - On command line, navigate to your project folder (maybe create `bootcamp-week3` folder to start Flask work). - Run: `python -m venv venv` (this creates a folder named `venv` containing the environment). - Activate it: on Windows `venv\Scripts\activate`, on macOS/Linux `source venv/bin/activate`. - You should see your prompt change (something like `(venv)` prefix). 2. **Install Flask:** Now run `pip install flask`. This downloads Flask (and its dependencies Werkzeug, Jinja2, etc.) into your venv. You can verify by running `pip freeze` to see installed packages. 3. **Project Structure:** Create a file `app.py` (our Flask app code will go here). You might also create subfolders for templates and static files later, but not needed for "Hello World".

Writing a Simple Flask App: In `app.py`:

```
from flask import Flask

app = Flask(__name__)    # create Flask application instance

@app.route("/")
def hello():
    return "Hello, Flask!"
```

Explain what's happening: - `Flask(__name__)` creates the app object. `__name__` tells Flask where to look for resources and templates, etc. It's a standard pattern. - The `@app.route("/")` is a **decorator** that associates the following function with the URL path `"/`" (the home page). That means when a browser goes to our site's root URL, this function will be called. - The function `hello()` returns

a string "Hello, Flask!" – Flask will convert that to an HTTP response. Here we return plain text; Flask by default will set a 200 OK status and content-type text/html for strings (the text will appear in browser). - We can add another route for practice, say:

```
@app.route("/greet/<name>")
def greet(name):
    return f"<h1>Hello, {name}!</h1>"
```

This shows a dynamic route: `<name>` in the route URL means this part is variable. If user goes to `/greet/Alice`, the `greet("Alice")` function is called and it returns a simple HTML header greeting Alice. If `/greet/Bob`, it returns greeting Bob. We're returning an HTML string here (h1 tag).

To run this application, at bottom of app.py add:

```
if __name__ == "__main__":
    app.run(debug=True)
```

- `app.run(debug=True)` starts the development server on default port 5000. `debug=True` is very useful in development: it auto-reloads the server on code changes and provides detailed error pages if something goes wrong. - Now, with your virtual environment activated, run `python app.py`. You should see output like:

```
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
```

- This means the server is running locally. Open a web browser and navigate to `http://localhost:5000/`. You should see "Hello, Flask!" displayed. Try `http://localhost:5000/greet/YourName` as well for the dynamic route.

Understanding Flask Output:

The string we returned became an HTML body. If you view page source, you'd see just `Hello, Flask!`. If you return HTML (like the `<h1>` in greet), the browser will render it accordingly (big bold text for h1). We could also return more complex HTML or even a full HTML page by returning a multiline string, but that's cumbersome – instead, Flask uses templates for that (coming soon).

Routing Concepts:

You can have multiple routes in one app. For example:

```
@app.route("/about")
def about():
    return "<p>This is a simple Flask application.</p>"
```

This would show a short message at `/about`. We should caution: if two routes have the same path (duplicate), Flask will override or error. Each route must be unique.

HTTP Methods in Flask:

By default, Flask routes respond to GET requests. If we want to handle form submissions (POST), we specify `methods` in the route decorator. E.g.:

```
@app.route("/submit", methods=["GET", "POST"])
def submit():
    if request.method == "POST":
        # process form data (we'll cover in Week 3)
        return "Form submitted!"
    else:
        return "<form method='post'>Name: <input name='name'><input
type='submit'></form>"
```

But we will delve into forms in Week 3. For now, just note how to allow methods.

Hands-On Lab – Flask Hello World:

If not already, follow along to: 1. Create and activate a virtual environment. 2. Install Flask. 3. Write the `app.py` with at least the `/` route. 4. Run the app and see it in a browser. 5. Add another route (like the `/about` or `/greet/<name>` examples) to practice. Save file, and because debug mode is on, the server will auto-reload. Refresh browser to see changes. 6. Try causing a small error to see debug mode: e.g., in one route, `1/0` (divide by zero) to simulate an error. Flask's debugger will show a stack trace in the browser (only use debug mode in development, not in production, as it can reveal code).

Guided Exercise – Routing Practice:

Extend your Flask app with a few more routes: - A route `/square/<int:x>` that takes an integer and returns its square. Flask can enforce type in the route (`<int:x>` ensures `x` is converted to int, and this route won't match non-integers). For example, `/square/5` should show "25".

```
@app.route("/square/<int:x>")
def square(x):
    return f"{x} squared is {x*x}"
```

- A route `/welcome` that reads a query parameter (e.g., `/welcome?name=Alice`). This involves using `request.args`. For now, just demonstrate:

```
from flask import request
@app.route("/welcome")
def welcome():
    name = request.args.get("name", "Guest")
    return f"Welcome, {name}!"
```

If you go to `/welcome?name=Alice`, it should say "Welcome, Alice!". If name param is missing, it uses "Guest" as default. - We will fully explain `request` object and query parameters in Week 3, but it's nice to see you can get data from the URL.

At this point, you have a basic Flask app running with multiple routes. This is the foundation: handling different URLs with Python code.

Closing notes for Day 4:

We've just scratched the surface of Flask. We can serve static content, but the real power is generating dynamic content, interacting with users via forms, and connecting to databases. We will build on this in Week 3 by introducing **templates** (to separate HTML from Python code) and user input handling. Also, recall the big picture: by end of bootcamp, you'll deploy a full Flask web app (with database, etc.). So keep this Hello World app as a starting point to expand.

Make sure you have your app.py and environment ready, as tomorrow we'll continue adding features to it. Commit your changes to Git as well (`git add app.py`, `git commit -m "Initial Flask app with basic routes"`) and push to GitHub if you have a repo.

Day 5: Week 2 Mini-Project – Build a CLI Address Book & Using GitHub Collaboration

Learning Objectives:

- Apply Week 2's topics (file I/O, OOP, and CLI/Git) in a cohesive mini-project.
- Design and implement a **Contact Manager (Address Book)** as a console application:
- Use a `Contact` class (from Day 3) to represent entries.
- Allow adding, listing, searching contacts via console menus.
- Store contacts in a file (so data persists between runs).
- Practice structuring a larger program and maybe using multiple modules (e.g., one module for contact class, one for main logic).
- Use Git for version control during development (multiple commits).
- Simulate collaboration: maybe one part of the code is "written by a teammate" and you pull changes, etc., to get experience.

This project will **not** use Flask (we'll have separate Flask projects). It's a console application to solidify core Python skills before we go all-in on web apps.

Project: Console Contact Book

Features: - The user can add a new contact (enter name, phone, email). - The user can list all contacts. - The user can search contacts by name. - (Optional) The user can delete a contact or update details. - Contacts are saved to a file (e.g., `contacts.csv` or `contacts.json`) so they remain after the program exits. - The program should load existing contacts from the file on start.

Plan the Data Format:

We can store contacts in a simple CSV (comma-separated values) text file, one contact per line as `name,email,phone`. Or use JSON for more structure. CSV is easier to manipulate with basic Python. Let's do CSV for this project.

Step 1: Define the Contact class (if not already):

```
class Contact:
    def __init__(self, name, phone, email):
        self.name = name
        self.phone = phone
        self.email = email
```

```
def __repr__(self):
    return f"{self.name} ({self.phone}, {self.email})"
```

We add a `__repr__` or `__str__` method to nicely print the contact. When we do `print(contact)` it will show Name (phone, email).

Step 2: File handling functions: We'll create two utility functions – one to load contacts from file at startup, and one to save contacts to file when needed.

```
import os

def load_contacts(filename):
    contacts = []
    if not os.path.exists(filename):
        return contacts # file not exist, return empty list
    with open(filename, "r") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            parts = line.split(",")
            if len(parts) == 3:
                name, email, phone = parts
                contacts.append(Contact(name, phone, email))
    return contacts

def save_contacts(filename, contacts):
    with open(filename, "w") as f:
        for contact in contacts:
            f.write(f"{contact.name},{contact.phone},{contact.email}\n")
```

Note: We decided to store as `name, email, phone`. The order is somewhat arbitrary; we should just be consistent (above, `Contact(name, phone, email)` but file is name,email,phone – oops, mismatched order for phone/email. Let's fix to store name, phone, email in that order for consistency): - Change file writing to `f.write(f"{contact.name},{contact.phone},{contact.email}\n")` and reading accordingly. Or adjust Contact initializer usage. We just need to be careful.

Step 3: Main program logic (menu): We will present a simple text menu to the user:

```
Contact Manager
1. List contacts
2. Add contact
3. Search contacts
4. Quit
```

Based on input, perform action.

Pseudo-code:


```

FILENAME = "contacts.csv"
contacts = load_contacts(FILENAME)

while True:
    print("\nContact Manager")
    print("1. List contacts")
    print("2. Add contact")
    print("3. Search contacts")
    print("4. Quit")
    choice = input("Choose an option: ").strip()
    if choice == '1':
        # list contacts
        if not contacts:
            print("No contacts found.")
        else:
            for idx, contact in enumerate(contacts, start=1):
                print(f"{idx}. {contact}")
    elif choice == '2':
        # add contact
        name = input("Name: ").strip()
        phone = input("Phone: ").strip()
        email = input("Email: ").strip()
        # Basic validation could be added (e.g., ensure not empty, etc.)
        new_contact = Contact(name, phone, email)
        contacts.append(new_contact)
        save_contacts(FILENAME, contacts)
        print(f"Contact {name} added.")
    elif choice == '3':
        # search
        term = input("Enter name to search: ").strip().lower()
        results = [c for c in contacts if term in c.name.lower()]
        if results:
            for contact in results:
                print(contact)
        else:
            print("No contacts found with that name.")
    elif choice == '4':
        print("Goodbye!")
        break
    else:
        print("Invalid option, please try again.")

```

We should run tests: - If `contacts.csv` doesn't exist, starting list is empty. - Adding contacts should create the file and save them. - Listing should show them with indices. - Searching should find partial matches (case-insensitive as written). - Quitting exits gracefully.

Improvement ideas (optional): - Add delete or update functionality. - Sort contacts by name when listing. - Use try/except to handle file errors or invalid input (like pressing Enter without any option). - Use CSV module or JSON for storage (CSV parsing manually is okay for now).

Testing the Application: Simulate usage:

```
> python contact_manager.py
Contact Manager
1. List contacts
2. Add contact
3. Search contacts
4. Quit
Choose an option: 1
No contacts found.
Choose an option: 2
Name: Alice
Phone: 111-2222
Email: alice@example.com
Contact Alice added.
Choose an option: 2
Name: Bob
Phone: 222-3333
Email: bob@example.com
Contact Bob added.
Choose an option: 1
1. Alice (111-2222, alice@example.com)
2. Bob (222-3333, bob@example.com)
Choose an option: 3
Enter name to search: ali
Alice (111-2222, alice@example.com)
Choose an option: 4
Goodbye!
```

Check the `contacts.csv` file – it should have two lines for Alice and Bob.

Version Control & Collaboration: Throughout development, you should be using Git: - Initialize a repo (if not done) and commit after implementing each part (after class definition, after file functions, after main menu working). - If working with a partner, one could implement `add contact` while another does `search contact`. One would commit and push, the other would pull and merge changes. If not actually collaborating, you can simulate by having a second fake contributor – but simply committing and pushing your own work is fine. - The key is to reinforce using Git regularly. At minimum, do a final commit: `git commit -am "Implement Contact Manager mini-project"` and push to your GitHub.

This project ties together many skills: reading/writing files (contacts saved to CSV), OOP (Contact class), loops and conditionals (menu system), string handling (search), and using external libraries (though we used core library here). It's a solid console program.

End-of-Week 2 Quiz:

1. Which Python keyword is used to handle exceptions?

- A. `try`
- B. `catch`

- C. `error`
- D. `except`

(Select two keywords that work together.)

2. What will the following code output?

```
class Person:
    def __init__(self, name):
        self.name = name
    def say_hi(self):
        print("Hi, I am", self.name)
p = Person("Alice")
p.say_hi()
```

- A. `Hi, I am Alice`
- B. `Hi, I am name`
- C. `Hi, I am`
- D. Error, `say_hi` not called correctly.

3. Which command creates a new Git repository in your current directory?

- A. `git start`
- B. `git init`
- C. `git commit`
- D. `git clone`

4. How do you open a file `data.txt` for appending in Python?

- A. `open("data.txt", "r")`
- B. `open("data.txt", "w")`
- C. `open("data.txt", "a")`
- D. `open("data.txt", "append")`

5. **Short Answer:** What is the purpose of a virtual environment in Python, and how do you activate one?

6. **Short Answer:** Explain what `@app.route("/")` does in a Flask application.

7. **Short Answer:** In Git, what is the difference between `git pull` and `git push`?

(End of Week 2 content. Weeks 3 and 4 to follow...)

Week 3: Web Application Development with Flask

Overview: Weeks 3 and 4 focus on building a full-stack web application using Flask. In Week 3, we will expand our Flask knowledge significantly. We'll introduce **HTML templates** and **Jinja2** for dynamic content, learn how to serve static files (CSS, images), and handle user input via **forms** (using HTTP POST). We'll also cover connecting our Flask app to a **database** – likely using SQLite for simplicity – so we can store data persistently on the server side. By the end of Week 3, you will have built a mini-

project (a basic web application, such as a blog or to-do list app) that has multiple pages, allows user input, and stores data. Week 4 will then focus on the capstone project and polishing skills like testing, documentation, and deployment.

Let's dive into the exciting world of web development with Flask!

Day 1: HTML & Jinja Templates in Flask

Learning Objectives:

- Understand the role of **templates** in Flask for separating HTML (presentation) from Python logic.
- Learn basic **HTML** structure and elements (enough to create simple pages for our app).
- Use **Jinja2** templating syntax (Flask's default template engine) to insert dynamic content into HTML.
- Set up a templates directory and create HTML template files.
- Render templates in Flask view functions using `render_template` function.
- Use template features: variables, control structures (`{% if %}`, `{% for %}` loops) to generate dynamic lists, etc.
- Serve a simple CSS file to style the pages (intro to static files in Flask).

What are Templates?

In Flask, instead of returning strings with HTML directly from view functions (which can get messy for anything beyond a trivial message), we use **template files** – which are basically HTML files with placeholders for dynamic content. This follows the MVC (Model-View-Controller) or similar separation principles: our Python functions act as controllers, templates are the view (UI), and data (model) comes from DB or logic.

Flask uses **Jinja2** as its template engine. Jinja allows us to write placeholders like `{{ variable }}` in HTML, which Flask will replace with actual values when rendering the template. It also allows control flow in templates with `{% ... %}` tags (for loops, if conditions, etc.).

Setting up Templates Directory:

By default, Flask looks for templates in a folder named **templates** in the same directory as your app script. Let's create a `templates/` directory next to `app.py`. Inside it, we'll make our first template, say `home.html`.

Basic HTML Refresher:

HTML is the structure of web pages. A minimal HTML5 page:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My Flask App</title>
  </head>
  <body>
    <h1>Welcome</h1>
    <p>Hello, world!</p>
```

```
</body>
</html>
```

We will embed dynamic parts in the body usually.

Creating a Template:

Example `home.html` in templates directory:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{{ title }}</title>
  <link rel="stylesheet" href="{{ url_for('static',
filename='styles.css') }}">
</head>
<body>
  <h1>Welcome to {{ title }}</h1>
  <p>{{ message }}</p>

  <ul>
    {% for item in items %}
      <li>{{ item }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```

This template expects three variables: `title`, `message`, and `items`. `title` is used both in the `<title>` tag and in the page header. `message` is a paragraph. `items` is presumably a list; we loop over it to create a list in HTML. The syntax: - `{{ ... }}` for expressions/variables to output. - `{% for ... %}` and `{% endfor %}` to loop. - Similarly, `{% if condition %} ... {% else %} ... {% endif %}` exists for conditionals in templates.

Also note: `<link rel="stylesheet"...>` uses `url_for('static', filename='styles.css')`. This is how we include a static CSS file. Flask serves files from a folder named **static** by default (we'll create one for CSS/JS/images). `url_for('static', filename='styles.css')` generates the correct URL for that file (like `/static/styles.css`).

Creating a Static Folder: Make a directory `static/` next to templates. Inside, create `styles.css`:

```
body { font-family: Arial, sans-serif; background-color: #f0f0f0; }
h1 { color: #2c3e50; }
```

Just some simple styling to see it works.

Using render_template in Flask: Now in `app.py`, instead of returning strings, import and use `render_template`:

```
from flask import Flask, render_template

@app.route("/")
def home():
    data = ["Item 1", "Item 2", "Item 3"]
    return render_template("home.html", title="My Flask App", message="Hello from Flask!", items=data)
```

What happens here: - Flask will find `home.html` in templates folder. - It will substitute `title` with "My Flask App", `message` with "Hello from Flask!", and `items` with the list provided. - The generated HTML string is returned as the response.

Start the app (make sure to set `app = Flask(__name__)` and `if __name__ == '__main__': app.run(debug=True)` as before). Visit `http://localhost:5000/`, you should see an HTML page with "Welcome to My Flask App" and the message, and a list of items. The background should be light grey (from CSS).

If the CSS doesn't load, check your link href or console for errors. It should be accessible at `/static/styles.css`. Make sure the static folder is recognized (by default, it is).

Template Inheritance (Optional Advanced):

If you have multiple pages that share a layout (like a base template with header/footer), Jinja supports template inheritance via `{% block %}` and extending. Given time, you might introduce: - Create `base.html` with common HTML structure and blocks to fill. - Other templates `{% extends 'base.html' %}` and override blocks.

However, if too much, we can keep it simple with separate full templates for each page in this bootcamp.

Jinja Syntax Quick Reference: - `{{ var }}` - output the value of var (HTML-escaped by default to prevent injection). - `{{ var|safe }}` - output var without escaping (if it contains HTML you want to render). - `{% ... %}` - for statements (loops, ifs, etc.). - Comments in templates: `{# this is a comment #}`.

Hands-On Lab - Dynamic Template: Using our app from Day 4: 1. Create `templates/` and `static/` folders. 2. Add a base template or directly create templates for your routes: - If you had a `/about` route returning text, make `about.html` to return instead. - Example `about.html`:

```
<h1>About This Site</h1>
<p>This site is a demonstration of Flask bootcamp project.</p>
```

And in `app.py`:

```
@app.route("/about")
def about():
    return render_template("about.html")
```

(We omit full HTML for brevity here, but ideally include html/head/body or use base template.) 3. Add some dynamic content to practice: For example, if you have a `/greet/<name>` route, have it use a template: - `greet.html` :

```
<h1>Hello, {{ name }}!</h1>
<p>Welcome to our site.</p>
```

- View function:

```
@app.route("/greet/<name>")
def greet(name):
    return render_template("greet.html", name=name)
```

Test that passing different names shows up properly.

1. Ensure your static CSS is applied to all pages: one approach is to use a base template or include the link in each template's head.

If base template route: `base.html` :

```
<!doctype html>
<html><head>
    <link rel="stylesheet" href="{{ url_for('static',
filename='styles.css') }}">
    <title>{% block title %}My Site{% endblock %}</title>
</head>
<body>
    <div id="nav">
        <a href="{{ url_for('home') }}">Home</a> |
        <a href="{{ url_for('about') }}">About</a>
    </div>
    {% block content %}{% endblock %}
</body></html>
```

Then `home.html` would start with `{% extends 'base.html' %}`, and inside define:

```
{% block title %}Home - My Site{% endblock %}
{% block content %}
<h1>Welcome</h1>
<p>{{ message }}</p>
{% endblock %}
```

This is advanced but shows how to avoid repeating nav, etc., in every page.

We likely simplify for beginners and not push too much inheritance if unfamiliar.

After Template Integration:

Your Flask app is now serving full HTML pages with dynamic data and static assets. This is a huge step toward a real web app.

Next Steps (Forms & Input):

Now that we can present pages nicely, the next big piece is handling user input via forms (like posting new data). That's for Day 2 and Day 3.

Day 2: User Input and Forms in Flask

Learning Objectives:

- Understand how HTML **forms** work and how to collect user input (text fields, submit button, etc.).
- Create forms in HTML templates and handle form submissions in Flask routes using **POST requests**.
- Use Flask's **request** object to get form data (`request.form`).
- Implement basic validation or feedback if needed (ensuring required fields, etc.).
- Preserve form inputs or display success messages by rendering templates after submission.
- (Optional) Flash messages in Flask for user feedback.
- Continue building a mini web application (e.g., a simple guestbook or to-do list) that allows user to submit new entries via a form, stores them (maybe in memory or file/DB), and displays them on a page.

HTML Form Basics:

An HTML form is a section of a page with input elements and a submit action:

```
<form action="/add" method="post">
  Name: <input type="text" name="name"><br>
  Message: <textarea name="msg"></textarea><br>
  <button type="submit">Submit</button>
</form>
```

Key attributes: - `action` - URL to send the form data to (often one of your routes). If omitted, defaults to current URL. - `method` - "GET" or "POST". GET will append inputs to URL (not suitable for sensitive or large data). POST sends data in the request body (preferred for forms modifying data). - Each input needs a `name` attribute; this is the key that will appear in `request.form`.

For example, if user enters name "Alice" and message "Hello", a POST request to /add happens with form data: `name=Alice`, `msg=Hello`.

Flask Side - Handling POST: In Flask route definitions, default methods is ["GET"]. To accept POST, specify:

```
@app.route("/add", methods=["GET", "POST"])
def add():
    if request.method == "POST":
```



```

# process form
name = request.form.get("name")
msg = request.form.get("msg")
# Typically, save the data (to list, database, etc.)
# Then redirect or render a new page
...
else:
    # GET request - perhaps show the form page
    return render_template("add_form.html")

```

`request.form` is a dict-like object of form fields. `.get()` helps avoid `KeyError` if missing.

Example Use-case – Guestbook: Let's say we want a simple guestbook: people can submit their name and a comment, and we display all entries on the homepage.

- We will keep guestbook entries in a list in memory for now (in a real app, you'd use a database).
- The homepage ("/") will show a list of entries and a link to "Add Entry".
- The "Add Entry" page ("/add", GET) will show a form.
- Submitting the form ("/add", POST) will add the entry and redirect back to homepage (to see the updated list).

Template for Form (add_form.html):

```

<h2>Sign the Guestbook</h2>
<form action="{{ url_for('add') }}" method="post">
  <p>Name: <input type="text" name="name" required></p>
  <p>Message: <textarea name="message" required></textarea></p>
  <p><button type="submit">Submit</button></p>
</form>

```

We use `required` attribute for basic HTML5 validation (ensuring not empty). `url_for('add')` gives the correct /add URL.

View function:

```

from flask import request, redirect, url_for

guestbook_entries = [] # global list to store entries

@app.route("/")
def index():
    return render_template("index.html", entries=guestbook_entries)

@app.route("/add", methods=["GET", "POST"])
def add():
    if request.method == "POST":
        name = request.form.get("name")
        message = request.form.get("message")
        if name and message:

```

```

        guestbook_entries.append({"name": name, "message": message})
        return redirect(url_for('index'))
    else:

# if missing data, you could re-render form with an error (simple handling)
        error = "Please enter both name and message."
        return render_template("add_form.html", error=error)
    else:
        # GET request: show the form
        return render_template("add_form.html")

```

We introduced `redirect` and `url_for` for after submission: - `redirect(url_for('index'))` sends a response telling browser to navigate to the home page. This is a common pattern after POST (Post/Redirect/Get) to avoid form resubmission issues. - Alternatively, we could render the index template directly with updated data, but using `redirect` is cleaner.

index.html should display entries:

```

<h1>Guestbook</h1>
<a href="{{ url_for('add') }}">Add a new entry</a>
{% if entries %}
    <ul>
        {% for entry in entries %}
            <li><strong>{{ entry.name }}</strong>: {{ entry.message }}</li>
        {% endfor %}
    </ul>
{% else %}
    <p>No entries yet. Be the first to sign!</p>
{% endif %}

```

This loops over entries and shows them. It's using each entry as a dict with 'name' and 'message'.

Test the flow: - GET / (no entries initially). - Click "Add new entry" (GET /add, shows form). - Submit with name and message (POST /add, server adds to list, redirects to /). - See the entry listed on homepage. - Add a second entry, etc.

Validation & Feedback: We did a simple check on server if fields are present, and in that case we could set an error message. In `add_form.html`, if we pass `error` in context:

```

{% if error %}<p style="color:red">{{ error }}</p>{% endif %}

```

This would show the error.

We can also use **flash messages**: Flask has `flask.flash()` to store a message in session and display it in next request, but that might be too advanced for now. Our approach is fine.

Protecting Forms (CSRF): In production, forms should use CSRF tokens for security, but that requires using Flask-WTF or manually implementing tokens. It's a bit advanced for this bootcamp level, so we'll

skip it. Just note: by default our form is vulnerable to cross-site request forgery. In a real app, you'd address that.

Hands-On Lab – Create a To-Do List App: Another idea for practice: allow adding tasks and listing them (similar pattern to guestbook). If guestbook done, skip or vice versa.

- Have a global list `todos = []`.
- Home page shows tasks (with a form to add new tasks on same page or separate).
- Or separate page /addtask to add.
- Show tasks with a checkbox perhaps (just display for now).
- This is similar to guestbook just different wording.

Testing and Committing: Try out invalid submissions (empty fields) to see error display. Try multiple entries.

Use Git: commit after implementing form page, commit after hooking it up, etc.

By the end of Day 2, you can: - Create forms in templates. - Handle POST in Flask, retrieve form data. - Update server-side data structures and redirect or render accordingly.

This is a major step: we can now have interactive web pages where users input information that our application processes and stores.

Day 3: Integrating a Database (Flask with SQLite)

Learning Objectives:

- Understand the need for a **database** instead of in-memory or file storage for persistent data in a web app.
- Learn basics of **SQL** and SQLite (a lightweight file-based database).
- Use **Flask extensions** or standard library to connect to SQLite.
- Set up a simple database schema (tables) for our app's data (e.g., for our mini project or upcoming capstone).
- Perform basic CRUD operations: Create, Read, Update, Delete using SQL queries in Flask.
- Possibly use **Flask-SQLAlchemy** (an ORM) as an alternative to raw SQL (if time permits), but might stick to raw SQL for clarity.
- Migrate our earlier guestbook or to-do app to use a database instead of a Python list, so data persists beyond app restarts and is multi-user safe.

Why a Database?

So far, we used a Python list or a file to store data. In a real server environment, when the app restarts, that list is cleared, and if multiple server instances, they don't share that list. A database solves persistent storage and concurrency. SQLite is a simple SQL database stored in a single file, perfect for small apps and testing.

SQLite Basics:

SQLite comes with Python as the `sqlite3` module. We can use it directly:

```
import sqlite3
conn = sqlite3.connect('app.db')
```

```

cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS entries (id INTEGER PRIMARY KEY, name TEXT, message TEXT)")
conn.commit()

```

This makes a table for guestbook entries with an auto-increment id.

We then use SQL commands for CRUD: - Insert: `INSERT INTO entries (name, message) VALUES (?, ?)`, using parameters to avoid SQL injection. - Select: `SELECT * FROM entries ORDER BY id DESC` (for example). - Delete or update if needed similarly.

Where to integrate in Flask app?

We can set up a database connection when the app starts or per request. Flask provides patterns: - Using application context and `g` object to store a connection. - Using Flask-SQLAlchemy extension which handles a lot automatically.

To keep it simpler: - We might establish a connection globally (not ideal for production concurrency, but fine here). - Or open and close on each database access.

For demonstration, let's do one connection per request pattern:

```

from flask import g

DATABASE = "app.db"

def get_db():
    if 'db' not in g:
        g.db = sqlite3.connect(DATABASE)
        g.db.row_factory = sqlite3.Row # to get dict-like row access
    return g.db

@app.teardown_appcontext
def close_db(exc):
    db = g.pop('db', None)
    if db:
        db.close()

```

What this does: - `get_db()` opens a connection and stores it in Flask's `g` (which is request-scoped). - `@app.teardown_appcontext` ensures after each request, if a connection was opened, it gets closed. - We use `sqlite3.Row` `row_factory` so we can treat query results like dictionaries (makes templating easier).

Initialize the Database (one-time): We need to create tables. We can either do it manually via a script or provide a route/command to initialize. For ease, we might do:

```

def init_db():
    db = get_db()
    db.executescript("""

```

```
CREATE TABLE IF NOT EXISTS entries (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    message TEXT NOT NULL
);
"""
```

Call `init_db()` at startup (if app is small, can just call right after defining it). Or better, use a flask CLI command, but that's extra.

Using the DB in our routes: Refactor guestbook: - On adding entry (POST /add): use `INSERT` instead of list append.

```
db = get_db()
db.execute("INSERT INTO entries (name, message) VALUES (?, ?)", (name,
message))
db.commit()
```

- On listing (GET /): use `SELECT`.

```
cur = get_db().execute("SELECT name, message FROM entries")
entries = cur.fetchall()
```

With `row_factory`, each `row['name']` will give name. Or iterate row as tuple if not using `row_factory`.

- We can pass `entries` (list of `sqlite3.Row` objects) directly to template. In Jinja, you can do `entry.name` or `entry['name']` if row factory used.

Complete Example:

```
@app.route("/")
def index():
    db = get_db()
    cur = db.execute("SELECT name, message FROM entries ORDER BY id DESC")
    entries = cur.fetchall()
    return render_template("index.html", entries=entries)

@app.route("/add", methods=["GET", "POST"])
def add():
    if request.method == "POST":
        name = request.form['name']
        message = request.form['message']
        if name and message:
            db = get_db()
            db.execute("INSERT INTO entries (name, message) VALUES (?, ?)",
(name, message))
            db.commit()
            return redirect(url_for('index'))
```

```
    else:
        error = "Both name and message are required."
        return render_template("add_form.html", error=error)
    else:
        return render_template("add_form.html")
```

This should have similar behavior as before, but now data persists in `app.db`. Test it: Run app, add entries, stop and restart server, entries should still be there (since stored in `app.db` file).

Flask-SQLAlchemy (Optional mention): Using raw SQL and connection is fine, but many prefer an ORM. Flask-SQLAlchemy simplifies by handling connections and giving a high-level model API. For brevity, we might not implement it fully, but can mention: - You'd define a class `Entry(db.Model)` with fields. - Then use `Entry.query.all()` to get entries, etc. Given bootcamp timeframe, likely skip detailed ORM usage.

Migrations: Also mention if schema changes, a migration tool is needed (Flask-Migrate or Alembic), but skip actual use.

If No DB: If this is too heavy, an alternative is to just save to a file (like we did for `contacts.csv` in week2). But we want them to experience using a database. SQLite is the simplest DB.

Hands-On Lab – Integrate DB in To-Do app: If they made a to-do list or similar app earlier, upgrade it to use SQLite: - Create tasks table. - On adding task, insert. - On listing, query DB. - Possibly allow marking complete: maybe have a checkbox form that triggers an update query. - This can get complex with forms for each task. Might leave update/delete as optional.

Focus on at least storing new tasks and listing tasks from DB.

Testing: Add some entries, ensure they display, check the actual DB file by opening with a SQLite browser or command-line.

Commit the changes to Git. Keep your app secrets out (no secrets here though as DB is local).

By end of Day 3: - You have a Flask app that interacts with a real database. This is a major step in creating full-stack apps. - You know how to structure code to open/close DB connection per request. - You can run SQL queries to manage app data.

This sets stage for Week 4 capstone where likely we'll build a more comprehensive app using these components (Flask + database + maybe user auth, etc., depending on scope).

Day 4: Finalizing the Capstone Project Plan and Best Practices

Learning Objectives:

- Plan the **Capstone Project**: decide on the application to build, features, and tasks breakdown.
- Outline the steps to build the capstone (which will be done in the remaining time).
- Discuss and implement additional **best practices**:
- Code organization: maybe using Blueprints in Flask if multiple modules.

- **Testing:** how to test parts of the application (could write a simple unit test or use Flask's test client for an endpoint).
- **Documentation:** writing a good README for the project, and comments or docstrings in code.
- **Code style:** ensure code is formatted (mention PEP8 and perhaps use a linter/formatter like Black).
- Ensure use of Git throughout the project, possibly working in branches or at least multiple commits.
- Start building capstone, or if already partly built from earlier weeks, improve it.

Capstone Project Introduction:

Now that we have acquired all the necessary skills, it's time to apply them in a **capstone project**. The capstone will be a full-stack Flask web application that you will design, implement, and deploy. We want it to incorporate: - Database for persistent storage. - Multiple routes (pages), templates for frontend. - Forms for user input. - Possibly an authentication system (user login) if manageable, or at least a well-structured multi-user scenario (like each user's own data). - Deployment instructions so it can be run on a server.

Choosing a Capstone App: Some ideas (choose one): - A **Blog** platform: Users can register, then create posts, edit/delete their posts, and view others' posts. (This involves user auth and relationships posts<->user.) - A **To-Do or Task Manager**: Users sign up and manage their personal to-do list (create, mark complete, etc.), possibly categorize tasks. - A **Simple Forum or Comment System**: People can post topics and comments. - A **Mini E-commerce-like app** (very scaled down): A product list and a cart (without actual payments). - Since our mini-project was guestbook, we could extend that into something with accounts. Given time, maybe a multi-user blog or task manager is doable.

For the sake of demonstration, let's assume we choose a **To-Do App with User Accounts**: - Users can register and log in. - Each user has their own tasks in a list. - They can add new tasks, mark tasks as done, and remove tasks. - Use a database to store users and tasks. - Possibly implement user sessions (login state). We can use Flask's `flask_login` extension or manual session handling.

Planning the Work (Daily tasks for Week 4): - Day 1 (today): Plan & DB models creation. - Day 2: Implement user registration and login (auth). - Day 3: Implement core app functionality (task CRUD). - Day 4: Testing, polishing, documentation. - Day 5: Deployment steps and wrap-up.

However, since in this outline Day 4 is planning and best practices, perhaps we've allocated: - Actually building might mostly happen Days 2-4, with Day 5 deployment.

Let's outline **Capstone requirements** clearly (fulfilling prompt: instructions, requirements, evaluation criteria): We will do this in the Capstone Project section after weekly breakdown.

Best Practices Discussion:

1. **Code Formatting & Style:**
2. Use consistent indentation (4 spaces).
3. Meaningful naming.
4. PEP8 highlights like limit line length ~79, etc. 4 9.
5. Possibly run a linter or formatter on your code (introduce Black, Flake8).
6. Encourage adding comments for tricky parts and docstrings for functions.

7. Testing:

8. We can write a few tests. For example, using Python's `unittest` or `pytest` to test a utility function or using Flask's test client to simulate requests:

```
from app import app
with app.test_client() as client:
    response = client.get('/')
    assert b"Welcome" in response.data
```

to test homepage returns something containing "Welcome".

9. Encourage at least writing tests for critical functions (like a function that adds two numbers - trivial example, or a form validation function).
10. Explain how automated testing ensures new changes don't break existing functionality.

11. Documentation:

12. At minimum, have a README.md in project describing what the app is, how to install/run, and an overview of features.
13. In code, include docstrings for modules, classes, and key functions explaining their purpose.
14. Possibly provide usage examples in docs.

15. Project Structure:

16. As app grows, consider organizing code: separate blueprint for auth vs main features, or at least separate Python files for routes vs models vs forms, etc.
17. If using Blueprints (Flask's way to structure app into components), mention it but maybe not fully implement given time.
18. Use `__init__.py` to initialize app and register blueprints.

We likely keep it simpler: one `app.py` for entire app due to scale, but mention that in bigger apps you'd break into multiple files.

Setting Up Version Control properly: - Perhaps create a GitHub repo specifically for capstone if not already. - Use branches if experimental features (one could try a 'dev' branch). - Ensure to commit at logical milestones. - Write commit messages that are descriptive.

Security Best Practices: - If implementing auth, store passwords hashed (mention not to store plain passwords, use `generate_password_hash` from `werkzeug.security`). - Use environment variables or config files for secrets (like secret key for sessions). - We won't dive deep but mention never commit actual credentials or secret keys to repo.

Prepare for Deployment: - For tomorrow, we'll discuss deploying (maybe to Heroku or an alternative). - Ensure `requirements.txt` is created (`pip freeze > requirements.txt`) so that dependencies are listed for deployment. - Possibly create a Procfile if deploying to Heroku.

Action for Day 4 (if any coding): - Possibly set up the database models for capstone. - If continuing from mini-project, migrate that data model and add user. - Or scaffold the project structure (create files, templates skeleton).

If capstone is from scratch, could start coding base. But likely building on what we have (like to-do with users).

Conclusion: By end of Day 4, we have a clear plan for capstone and are mindful of writing quality code. We might begin some implementation so that Day 5 is not too rushed (depending on actual time, though Day 5 is said to be deployment and final touches).

Day 5: Deployment and Capstone Project Submission

Learning Objectives:

- Learn how to **deploy** a Flask application to a server or platform (e.g., deploying to Heroku, PythonAnywhere, or similar).
- Prepare the project for production: setting debug False, handling static files, etc.
- Write deployment instructions (so user can replicate).
- Finalize the capstone project: ensure all required features are implemented and working.
- Go through the **evaluation criteria** and ensure the project meets each point:
 - Correctness of functionality.
 - Code quality (style, comments).
 - Completeness of features.
 - Documentation and ease of use.
- Submit or present the project.

Deployment Options: As of 2025, common and (hopefully free-tier) options: - **Heroku** (though free tier removed in 2022, but maybe this is an educational scenario; or can mention GitHub Student Pack might give credits). - **Render** or **Railway** – similar PaaS. - **PythonAnywhere** – has a free tier for simple apps. - **Vercel** or **Netlify** can also host Python via serverless functions, but more advanced.

We will show one process, e.g., deploying to **Render** (quickly if possible), or use **PythonAnywhere** which is simpler with web interface.

Heroku Deployment Steps (if using): 1. Sign up and install Heroku CLI. 2. Add a `Procfile` with `web: gunicorn app:app` (gunicorn to run WSGI server). 3. Ensure requirements.txt is up-to-date. 4. `heroku create` and `git push heroku main`. 5. Set config vars if needed (like `FLASK_ENV=production`). 6. Open the app.

But since no free Heroku, maybe **Render**: - Connect GitHub repo to Render, create a web service, specify start command (gunicorn as above), it auto-deploys.

PythonAnywhere: - Create account. - Upload files or use `git clone` from their bash console. - In the dashboard, set up a new Web App (choose Flask, Python version). - It will make a default app, then you edit WSGI file to point to your app. - Install any requirements via pip in console. - Reload web app, then access on provided domain.

Given constraints, maybe PythonAnywhere (free and easy) is a good choice for beginners: Detailed:

1. Go to PythonAnywhere, create a free account.
2. Start a "New Web App", choose manual configuration for Flask.
3. Upload your project files using the file uploader, or better:
 - Open a Bash console on PythonAnywhere.
 - ``git clone https://github.com/YourUser/your-repo.git`` (if repo is public or use token if private).
4. In the Web tab, set the working directory to your app folder, and point the WSGI configuration to your Flask app (the WSGI file needs to have something like:

```
import sys
path = '/home/youruser/yourproject'
if path not in sys.path:
    sys.path.append(path)
from app import app as application
where app is your Flask instance).
```
5. Install dependencies: in console, ``pip install -r requirements.txt``.
6. Reload the web app via the PythonAnywhere interface.
7. Visit your site (username.pythonanywhere.com).

We provide these instructions possibly.

Things to consider in production: - Turn off debug mode (never expose debug in public). - Use an actual server like gunicorn for production (if on Heroku/Render, gunicorn is used). - Set a secret key for session (should not be default). - If using a database, if multi-user concurrency, maybe use a more robust DB or at least be careful with SQLite (it's fine for low traffic though).

For our scope, it's okay.

Wrapping up Capstone:

Make sure the project: - Meets all features specified. - Has no glaring bugs. - The code is cleaned (remove prints or test code). - Possibly add any final enhancements if time (like client-side form validation or a nicer UI, e.g., using a CSS library like Bootstrap by including its CDN link). - Write a README that includes: - Project overview. - How to install and run (including dependencies and possibly how to init DB). - Features and usage guide. - Attribution if any code/template taken from somewhere.

Evaluation Criteria: We will list in the final output bullet points such as: - *Functionality*: Does the app meet specified requirements (features X, Y, Z)? - *Code Quality*: Is the code well-structured, readable, and following style guidelines? (No obvious code smells, proper formatting ¹⁰). - *User Experience*: Is the app easy to use, with clear instructions, and proper feedback for actions? (E.g., error messages, confirmation of actions). - *Documentation*: Is there adequate documentation (comments, README, possibly inline docs for complex functions)? Is setup/deployment explained? - *Git usage*: Has version control been used effectively? (Many commits with meaningful messages, maybe evidence of branches or pull requests if collaborative). - Possibly *Originality/Creativity*: if someone implemented extra features or a nice design beyond minimum. - *Completion*: Does the project have all required components (database integration, form handling, etc.)?

We instruct that you should test your project as if you are the end-user and try to break it, then fix issues.

Finally, encourage them to be proud of their accomplishment – from knowing nothing to deploying a web app in 1 month is huge!

Now we'll write the final "Capstone Project" section including instructions, requirements, evaluation criteria.

Capstone Project: Full-Stack Flask Web Application

Project Description:

For your capstone, you will build a full-stack web application using Flask that incorporates everything you've learned. The app should include user interaction (forms), persistent data storage (database), and dynamic content rendered with templates. You are free to choose the specific theme of your app (e.g., a blog, a task manager, a small e-commerce prototype, etc.), but it must meet the requirements below.

Requirements and Features:

- **User Accounts:** Implement a user registration and login system so that each user can have their own data on the app. Use secure password handling (store hashed passwords, not plain text). Upon login, use session or Flask-Login to keep the user logged in.
- **CRUD Functionality:** The app should allow Create, Read, Update, Delete operations on at least one main feature. For example, in a blog app – users can create posts, view a list of posts, edit their posts, and delete posts. In a task app – users can add tasks, view their task list, mark tasks complete, and remove tasks.
- **Flask & Database Integration:** Use Flask with an SQLite database (or another database) to store data. Define a clear schema for your data (tables and fields). Access the database from your Flask routes to save and retrieve information. Ensure that data is properly associated with the logged-in user when applicable (e.g., a user should only see their own to-do items).
- **Templates & Styling:** Use Jinja2 templates to render HTML pages. Include at least one base layout that other pages extend (to maintain consistent header/footer or navigation). The app should have a simple, clean user interface. Use a CSS framework (like Bootstrap) or custom CSS for basic styling so that the app is visually presentable.
- **Forms & Validation:** Provide HTML forms for user input (registration, login, creating posts/tasks, etc.). Validate form data on the server side (and client side with HTML5 or JS if possible). Give users feedback on validation errors (e.g., display an error message if a required field is empty or password is too short).
- **User Feedback & UX:** Utilize Flask's flashing system or equivalent to show success/error messages (for example, "Post created successfully" or "Invalid credentials" upon login failure). Navigation should be clear (include a navigation bar or menu to move between pages like Home, Create New, Profile, etc.). If an action requires login (protected page), ensure the app redirects appropriately to the login page.
- **Code Organization:** Organize your code logically. For example, you might separate your Flask app setup from routes by using Blueprints or separate modules for routes, models, forms, etc. (in a simple project, all routes can be in app.py, but structure is a bonus). Include comments and docstrings to explain complex sections. Avoid deeply nested or very lengthy functions – refactor into helper functions where appropriate.
- **Testing:** Include at least a few basic tests. This could be unit tests for a utility function or using Flask's test client to check that certain pages load (HTTP 200) and that a new item creation

actually adds to the database. You don't need exhaustive coverage, but demonstrate you know how to write a test or two.

- **Documentation:** Write a **README.md** for your project. It should contain:
 - Project title and description.
 - Setup instructions (how to install requirements, initialize the database, and run the app).
 - Usage instructions (how to create an account, use the features).
 - Any known issues or future improvements.
- Your name and perhaps contact or portfolio info (this is your project—you may want to show it off!).
- **Deployment:** Deploy your application to a hosting service (e.g., PythonAnywhere, Heroku, Render). Ensure that the deployed version is accessible and functional. Include the deployment link in your README. The deployed app should have DEBUG turned off and handle static files correctly. (It's okay if some features like email sending are not enabled due to host limitations, but core features should work). *If deployment is not possible, be sure to clearly document how to run the app locally and demonstrate it working.*

Project Development Tips:

- **Phase 1 – Database and Models:** Design your database schema first. Determine what tables you need (users, and one or more tables for your main data). Create a database initialization routine (you can make a small Python script or Flask CLI command to create tables). Test that you can connect to the database and add/read a sample record.
- **Phase 2 – User Authentication:** Implement registration and login functionality early. Flask-Login can simplify session management. Users should be able to create an account (store username/email and hashed password in DB) and then log in with those credentials. Protect routes that require login – e.g., if a non-logged user tries to access `/new_post`, redirect them to login. Once this is in place, it will simplify testing other features as a logged-in user.
- **Phase 3 – CRUD Features:** Implement the core feature of your app (posts, tasks, etc.). Start with the ability to create and read items. For example, make a form to create a new blog post, save it to DB with the current user's ID, and then show the post on a posts listing page. Then add update (maybe an "Edit" button on each item) and delete (a "Delete" button). Ensure only the owner of an item can edit or delete it (check user session vs item's user_id). Use confirm dialogs or separate confirmation pages for deletions to avoid accidental deletes.
- **Phase 4 – Polish UI and UX:** Add navigation links (Home, Create new, Logout, etc.). Use Bootstrap or another CSS library to quickly improve the look (link via CDN in base template). Ensure forms have proper labels and that required fields have `required` attribute for client-side enforcement. Implement flash messages for user actions (e.g., "Task added!" after adding a task, or "Logged in successfully" on login).
- **Phase 5 – Testing and Debugging:** Manually test all user flows:
 - Can a new user register and then log in?
 - What happens if they try a wrong password?
 - Can they create a new item? Does it appear immediately?
 - Can they edit and delete items, and do those changes reflect in the UI and DB?

- If you try to edit someone else's item by manipulating URL, does the app prevent it? (It should – implement a check or hide those options.)
- Does the app gracefully handle invalid input (like missing fields, or extremely long text)?
- Do pages load without errors (check browser console for 404s on resources or other errors)?
- **Phase 6 – Documentation and Deployment:** Ensure your code is clean (remove print/debug statements, ensure sensitive config like secret keys or DB credentials are not hard-coded in public code – use environment variables or a config file). Write the README as described. Then follow deployment steps to publish your app. For example, on PythonAnywhere, set up the web app, or on Heroku, push your code and make sure to set up any config vars (like `FLASK_ENV=production` and a secret key). Test the live site as a user once deployed.

Evaluation Criteria:

When you submit your capstone, it will be evaluated on the following points:

- **Completeness:** All required features are implemented. The app is functional for its intended purpose (users can register, log in, and perform the main tasks of the app). CRUD operations work correctly. Data is actually persisted in the database.
- **Correctness & Reliability:** The application runs without crashing or major bugs. Forms properly validate input and handle error cases (e.g., showing error if input is invalid). Security measures appropriate to a beginner project are in place: passwords hashed, login required for sensitive actions, etc. The logic enforcing permissions and data integrity is correct (e.g., one user cannot alter another's data via forging requests).
- **User Experience (UX):** The app is user-friendly. Navigation is intuitive (menus or links are provided, no dead-end pages). UI is reasonably styled – does not need to be fancy, but HTML shouldn't be overly broken or hard to read. Messages and labels are clear to the user. Feedback is given for actions (e.g., "Saved changes" message or highlighting form errors).
- **Code Quality:** The code is well-organized and readable. Follows Python style guidelines (proper indentation, descriptive variable and function names, not too long lines) ¹⁰ ⁹ . There is minimal duplicate code (utilize functions/loops to avoid copy-pasting). Comments or docstrings are present to explain non-obvious parts. If multiple files/modules are used, the separation of concerns is logical (e.g., a module for routes, a module for database models, etc.).
- **Use of Tools and Workflow:** You have used Git for version control with a meaningful commit history (we're not looking for a commit after every character, but there should be multiple commits representing progressive milestones of the project, each with a clear message). If collaboration was part of the project, proper use of branching or pull requests can be a plus. The README and documentation exist and make it easy to understand and run the project. If tests are included, that's a bonus demonstrating good development practice (we will run them if provided).
- **Deployment & Presentation:** The project is deployed or easily deployable following your instructions. We can access the live application or run it locally without undue difficulty. The README's instructions should yield a working local instance (assuming we follow steps and prerequisites). The overall presentation of the project (both in code and in app UI) shows pride

and effort – think of it as something you could show a potential employer or include in a portfolio.

Congratulations on reaching this point! The capstone project is your opportunity to synthesize everything from this bootcamp into one coherent application. Take it step-by-step, and don't hesitate to refer back to earlier course material (weeks 1-3 content) when implementing specific features. Good luck, and have fun – once deployed, you will have a web application you built from scratch available for the world (or at least your friends) to use!

1 2 **Python | One Month**

<https://onemonth.com/courses/python>

3 **PEP 8 - Codefinity**

<https://codefinity.com/blog/PEP-8>

4 5 9 10 **PEP 8 – Style Guide for Python Code | peps.python.org**

<https://peps.python.org/pep-0008/>

6 **Git Tutorial**

<https://www.w3schools.com/git/>

7 **State the core components of an HTTP response ? - GeeksforGeeks**

<https://www.geeksforgeeks.org/software-engineering/state-the-core-components-of-an-http-response/>

8 **Welcome to Flask — Flask Documentation (3.1.x)**

<https://flask.palletsprojects.com/en/stable/>